



CPCS 211: Digital Logic Design

Chapter 6 : Analysis of Sequential Systems

Outline

- State Tables and Diagrams
- Latches
- Flip Flops
- Flip flops with Clear and Preset Inputs
 - SR Flip Flop
 - JK Flip Flop
 - T Flip Flop
- Analysis of Sequential Systems

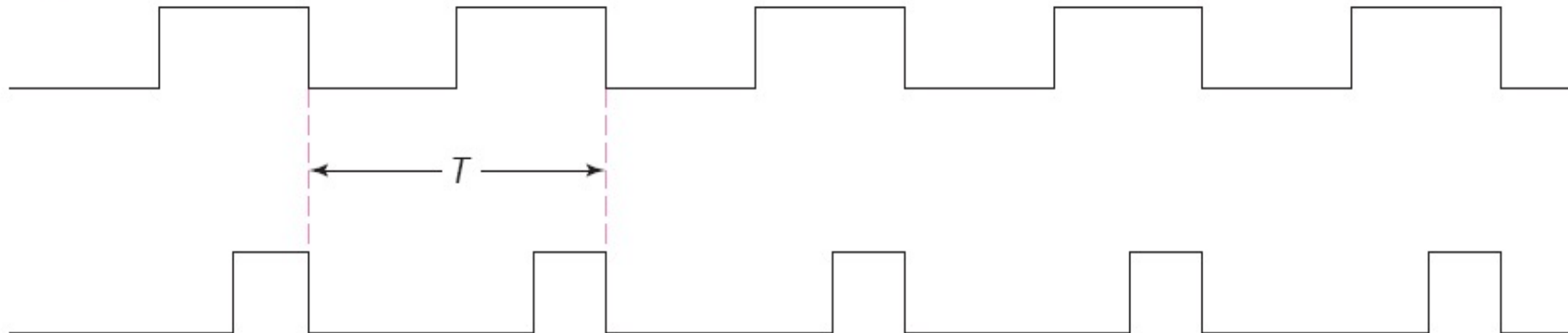
Sequential Systems

- Systems that have memory, are referred to as
 - Sequential systems, or as
 - Finite state machines
- In these systems
 - Output will depend not only on the **present** input but also on the **past**
- Here, we will deal primarily with *clocked systems*
 - Sometimes referred to as *synchronous systems*

Clock Signals

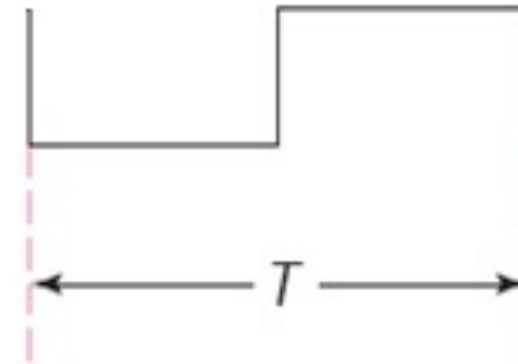
- A clock is just a signal that alternates (over time) between 0 and 1 at a regular rate
- Two versions of a clock signal are shown in Figure 6.1
 - In the first, clock signal is 0 half of the time and 1 half of the time
 - In the second, clock signal is 1 for a shorter part of the cycle

Figure 6.1 Clock signals.



Clock Signals

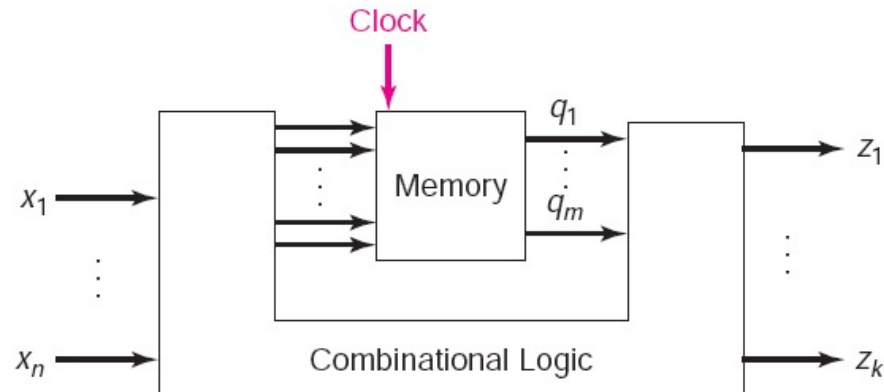
- *Period* of the signal (T on the diagram) is the length of one cycle
- *Frequency* is the inverse ($1/T$) of time period
 - A frequency of 200 MHz (megahertz, million cycles per second) corresponds to a period of
 - 5 nsec
 - 5 nanoseconds
 - 5 billionths of a second



Sequential Systems

- In most synchronous systems, change occurs on the transition of the clock signal
- Block diagram of Figure 6.2 is a conceptual view of a synchronous sequential system

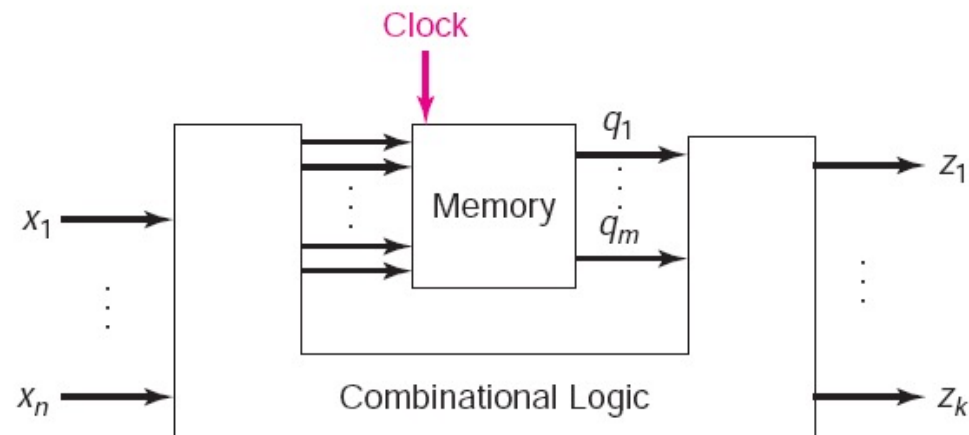
Figure 6.2 Conceptual view of a sequential system.



Sequential Systems

- A sequential system consists of a set of
 - memory devices and
 - some combinational logic
- Diagram depicts a system with n inputs (x 's), in addition to clock (k), outputs (z 's), and m binary storage devices (q 's)

Figure 6.2 Conceptual view of a sequential system.



Sequential Systems

- Each memory device may need one or two input signals
- Many systems have only one input and one output
- Many memory devices provide the complemented output q' as well as q
- Inputs to the combinational logic are the system inputs and the contents of memory
- Combinational logic outputs are the system outputs and signals to update the memory

System State

- Consider a system with one input x and one output z such that $z = 1$ iff x has been 1 for at least three consecutive clock times
- For this example
 - System must store, in its memory, information about last three inputs
 - to check whether they are 1 or not
 - and produce an output based on that
- What is stored in memory is the *state* of the system

System State

- Memory consists of a set of binary devices
 - They may just store the last few inputs, but it is often more economical to code the information in a different way
- Sometimes, a finite number of recent inputs is not adequate

Timing Trace

- *Timing trace* is a set of values for *input* and *output* (and sometimes *state* or other variables of system, as well) at consecutive clock times
- Often used to clarify the definition of system behavior or to describe the behavior of a given system
- Inputs are an arbitrary set of values that might be applied to the system, chosen so as to demonstrate the behavior of the system

Moore Model

- For the previous system, three consecutive 1's, timing trace is shown
- Output depends only on the state of the system (not the present input) and, thus, occurs after the desired input pattern has occurred
- Such a system is called **Moore** model

Trace 6.1 Three consecutive 1's.

x	0	1	1	0	1	1	1	0	0	1	0	1	1	1	1	1	0	0			
z	?	0	0	0	0	0	0	1	0	0	0	0	0	0	1	1	1	0	0	0	0

Timing Trace

- Output for the first input is shown as unknown (?)
 - Because we have no history of what happened before
 - If system were initialized to indicate that no 1's had yet occurred, then that output would be 0
- After three consecutive inputs are 1, the system goes to a state where the output is 1 and remains there as long as the input remains 1

Trace 6.1 Three consecutive 1's.

x	0	1	1	0	1	1	1	0	0	1	0	1	1	1	1	1	0	0		
z	?	0	0	0	0	0	0	1	0	0	0	0	0	0	1	1	1	0	0	0

State Table and State Diagram

For describing sequential systems, we use the following two tools:

- 1) **State table** shows, for each input combination and each state, what the **output** is and what the **next state** is
 - i.e., what is to be stored in memory after the next clock

- 2) **State diagram** (or state graph) is a graphical representation of the behavior of the system, showing for each input combination and each state what the **output** is and what the **next state** is
 - i.e., what is to be stored in memory after the next clock

Present and Next State

- We will refer to the present state as q and the next state as q^*
 - Some books use Q or q^+ or $q(t + \Delta)$ to represent the next state
- *Next state* is what will be stored in memory after this clock transition
 - That will then become the present state at the next clock time
 - Next state is a function of the present state and the input, x
- Output, in this example
 - Depends on the present state, but not the present input
 - Only changes when state changes, at time of a clock transition

State Table

Table 6.1 shows an example of a state table, one that does describe the previous system, although that is not obvious at this point

Table 6.1 A state table.

Present state	Next state		Present output
	$x = 0$	$x = 1$	
A	A	B	0
B	A	C	0
C	A	D	0
D	A	D	1

State Table

- First row of table signifies that
 - If system is in state *A*, that is, fact *A* is stored in memory and input is a *0*, then, next state is *A* (that is, *A* is to be stored in memory again); and
 - If fact *A* is stored in memory and the input is a *1*, then next state is *B*
- Whenever system is in state *A* (or *B* or *C*), output is *0*

Table 6.1 A state table.

Present state	Next state		Present output
	<i>x</i> = 0	<i>x</i> = 1	
<i>A</i>	<i>A</i>	<i>B</i>	0
<i>B</i>	<i>A</i>	<i>C</i>	0
<i>C</i>	<i>A</i>	<i>D</i>	0
<i>D</i>	<i>A</i>	<i>D</i>	1

State Diagram

State diagram that corresponds to the previous state table is shown

- Each state is represented by a *circle*
- Included in the circle is *output* for that state
- Each *line* coming out of circle represents possible transition
- *Label* on a line indicates input that causes that transition
- There must be *one path* from each state for each possible input combination

Figure 6.3 A state diagram.

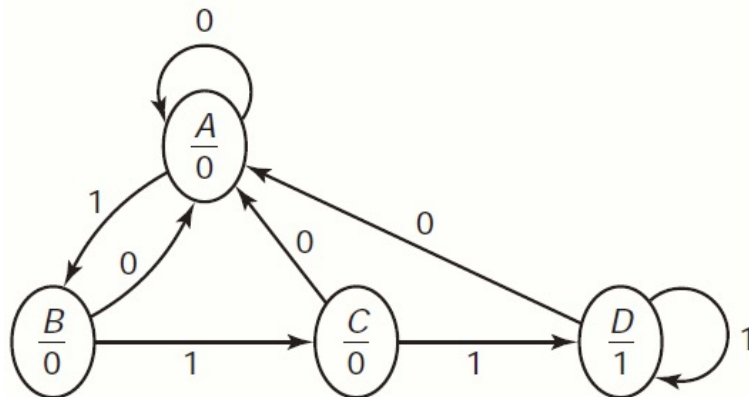


Table 6.1 A state table.

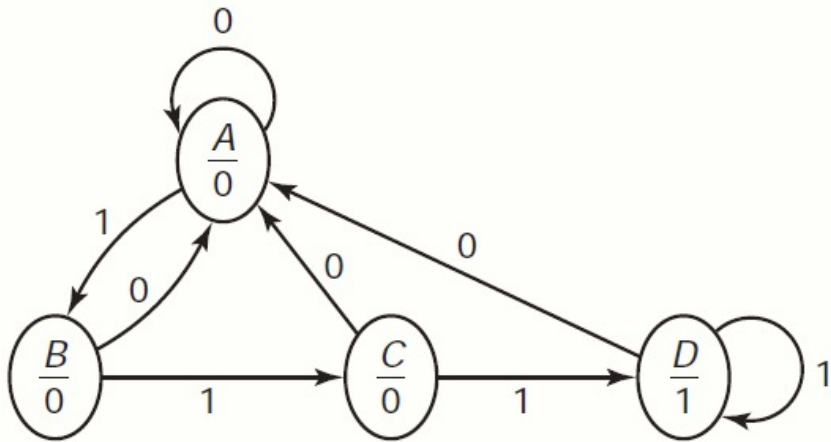
Present state	Next state		Present output
	$x = 0$	$x = 1$	
A	A	B	0
B	A	C	0
C	A	D	0
D	A	D	1

Trace with State

If we were given state table or diagram, we can construct timing trace

- Previous trace is repeated, but includes state

Figure 6.3 A state diagram.



Trace 6.2 Trace with state.

x	0	1	1	0	1	1	1	0	0	1	0	1	1	1	1	1	0	0		
q	?	A	B	C	A	B	C	D	A	A	B	A	B	C	D	D	D	A	A	?
z	?	0	0	0	0	0	0	1	0	0	0	0	0	0	1	1	1	0	0	0

Trace with State

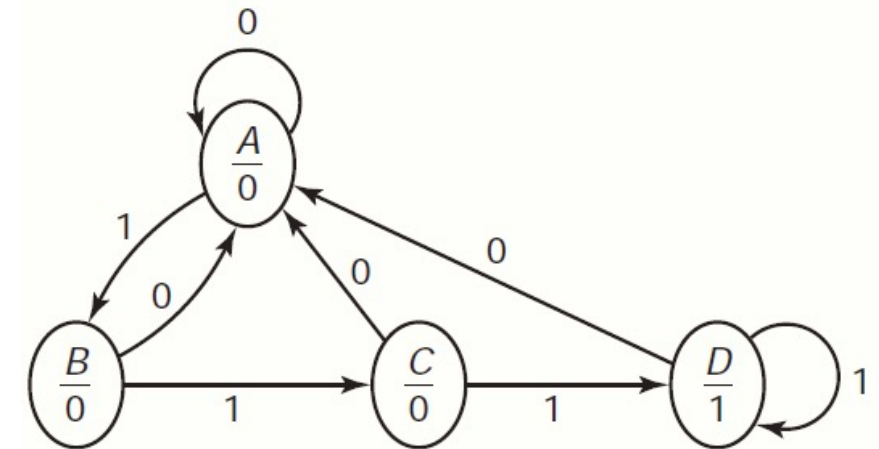
State table and state diagram both show that

- 0 input takes system to state *A* from all states
- 1 input takes system
 - from state *A*, to state *B*
 - from *B*, it goes to *C*
 - from *C*, it goes to *D*
 - from *D*, it remains in *D*

Trace 6.2 Trace with state.

x	0	1	1	0	1	1	1	0	0	1	0	1	1	1	1	1	0	0		
q	?	A	B	C	A	B	C	D	A	A	B	A	B	C	D	D	D	A	A	?
z	?	0	0	0	0	0	0	1	0	0	0	0	0	0	1	1	1	0	0	0

Figure 6.3 A state diagram.



Mealy Model

- In some systems, the output depends not only on the **present state** of the machine, but also on the **present input**
- This type of system is referred to as a **Mealy model**
- State table has as many output columns as the next state portion (one for each possible input combination); Example is shown

Table 6.2 State table for a Mealy model system.

q	q^*		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	A	B	0	0
B	A	C	0	0
C	A	C	0	1

Mealy Model

- Mealy model state diagram is different from Moore model
- Here, output is associated with **transition**, rather than state, as shown

Figure 6.4 State diagram for the Mealy system of Table 6.2.

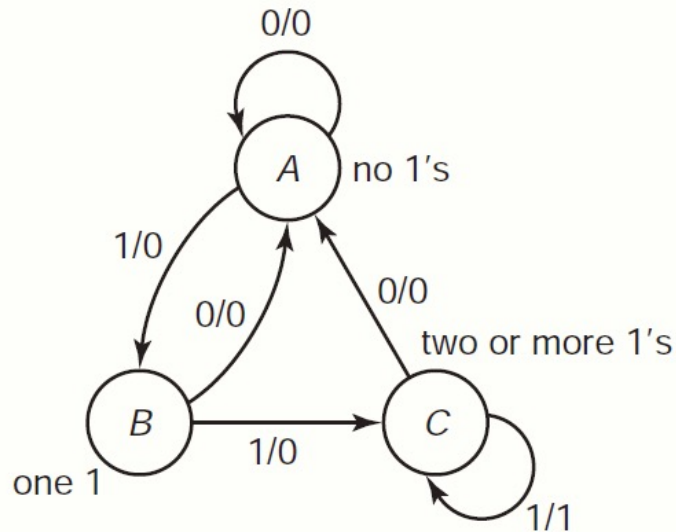
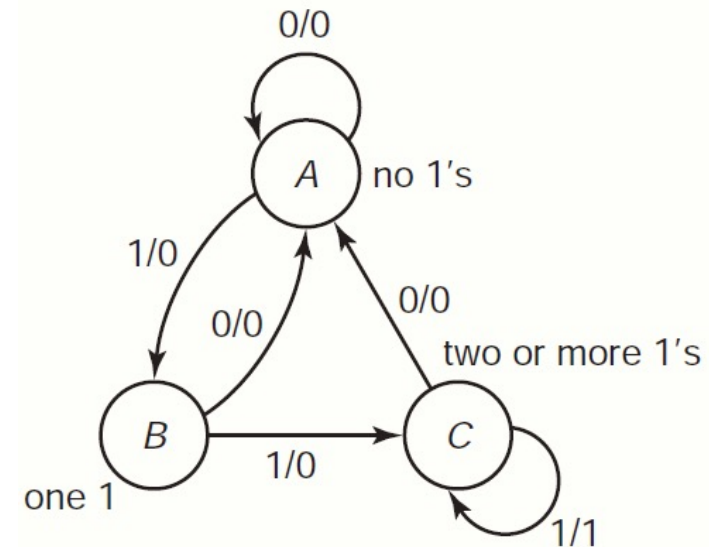


Table 6.2 State table for a Mealy model system.

q	q^*		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
A	A	B	0	0
B	A	C	0	0
C	A	C	0	1

Mealy Model

- Each path has a double label
 - *input* causing the transition followed by a *slash*
 - *output* that occurs when the system is in that state and that is the input
- Thus, from state *A*, the path to state *B* is labeled *1/0*, meaning that
 - that path is followed when $x = 1$
 - output produced is *0*

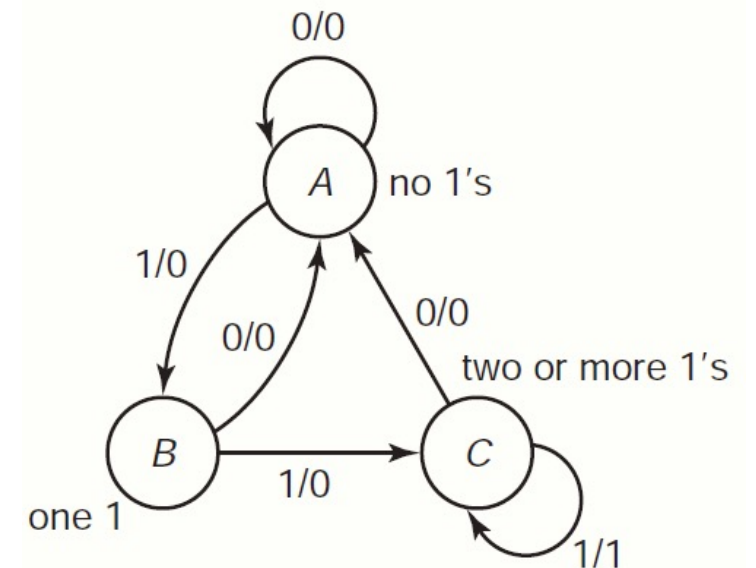


Timing Trace

- Timing trace (Trace 6.3) is constructed in much the same way as for a Moore model, except that the present output depends on the present input as well as the present state
- Analysis and design process for Moore and Mealy machines are very similar

Trace 6.3 Timing trace.

x	0	1	1	0	1	1	1	0	0	1	0	1	0	1	1	1	1	1	0	0		
q	?	A	B	C	A	B	C	C	A	A	B	A	B	A	B	C	C	C	C	A	A	
z	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0

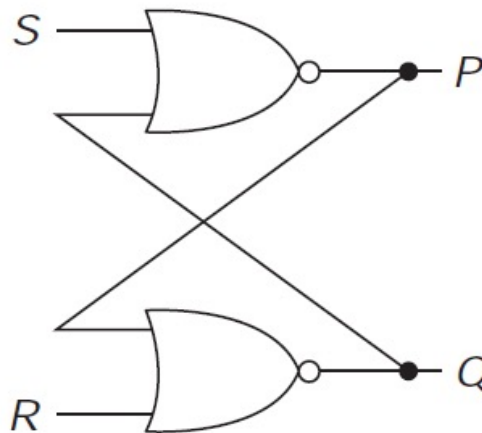


Latches and Flip Flops

Latches

- A *latch* is a binary storage device, composed of two or more gates, with feedback
 - That is, for the simplest two-gate latch, output of each gate is connected to input of the other gate
- Figure 6.5 shows such a latch, constructed with two NOR gates

Figure 6.5 A NOR gate latch.



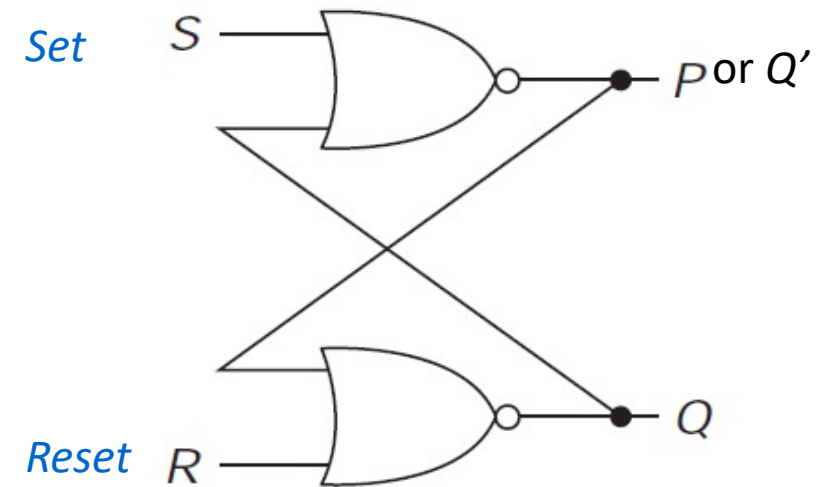
Latches

Equations for this system

$$P = (S + Q)'$$

$$Q = (R + P)'$$

Figure 6.5 A NOR gate latch.



Latches

- The normal storage state is both inputs 0 (*inactive*)
- If S and R are 0, then both equations state that P is opposite of Q , i.e.,
- The latch can store either
 - 0 ($Q = 0$ and $P = 1$) or
 - 1 ($Q = 1$ and $P = 0$)
- Thus, output P is usually just labeled Q'

Latches

- Letter *S* is used to indicate *set*, i.e., store a *1* in the latch
- Letter *R* is used to indicate *reset*, i.e. store a *0* in the latch
- To analyze the operation of the previous *SR-latch* with *NOR* gates

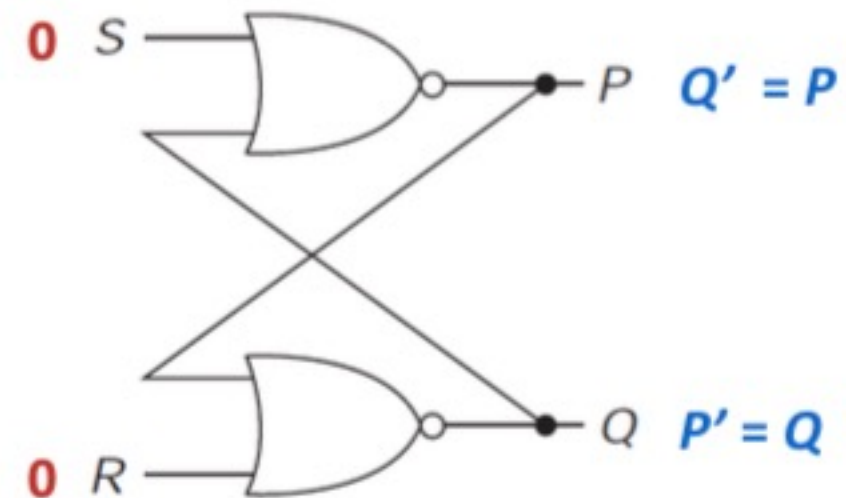
Remember that output of *NOR* gate is

- *0* if any of the inputs is *1*
- *1* only when all inputs are *0*

SR-Latch

If $S = 0$ and $R = 0$

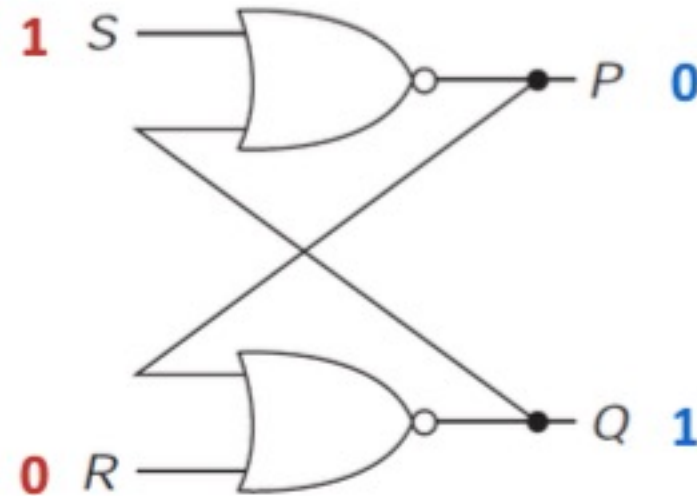
- $P = (0 + Q)' = Q'$
- $Q = (0 + P')' = P'$
- P' is stored in the latch (on line Q)
- no change



SR-Latch

If $S = 1$ and $R = 0$

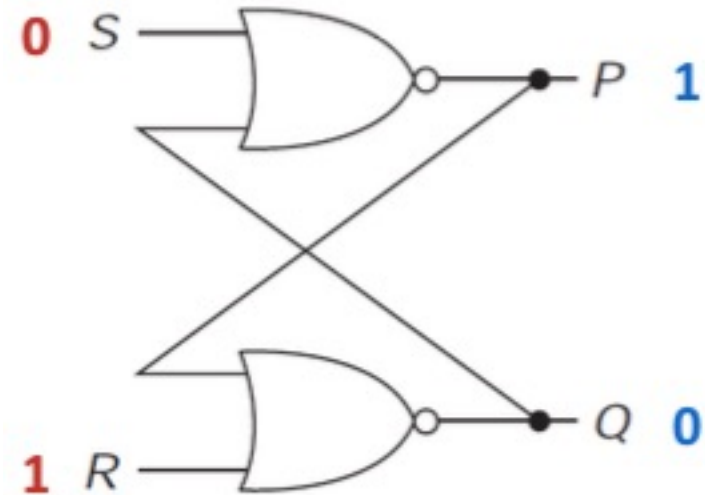
- $P = (1 + Q)' = 1' = 0$
- $Q = (0 + 0)' = 0' = 1$
- 1 is stored in the latch (on line Q)
- set



SR-Latch

If $R = 1$ and $S = 0$

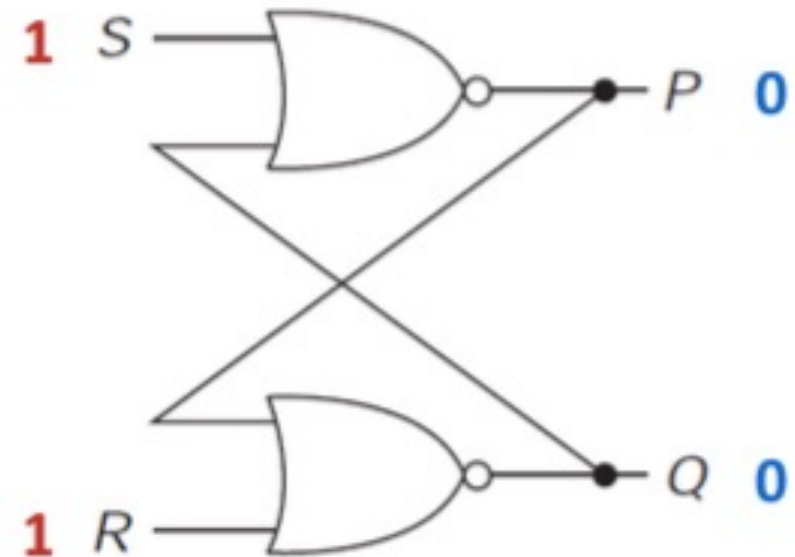
- $Q = (1 + P)' = 1' = 0$
- $P = (0 + 0)' = 0' = 1$
- 0 is stored in the latch (on line Q)
- reset



SR-Latch

Latch is not operated, if $S = 1$ and $R = 1$

- $P = (1 + Q)' = 1' = 0$
- $Q = (1 + P)' = 1' = 0$
- $P = Q!!$
- not operated (indeterminate)

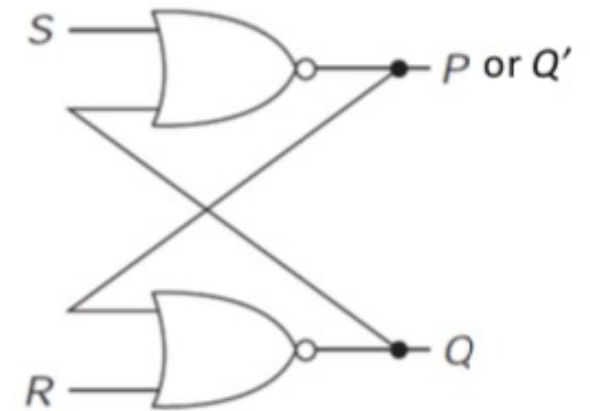


SR-Latch

- If $S = 1$ and $R = 0$
 - $P = (1 + Q)' = 1' = 0$
 - $Q = (0 + 0)' = 0' = 1$
 - 1 is stored in the latch (on line Q)
- If $R = 1$ and $S = 0$
 - $Q = (1 + P)' = 1' = 0$
 - $P = (0 + 0)' = 0' = 1$
 - 0 is stored in the latch (on line Q)
- Latch is not operated, if $S = 1$ and $R = 1$
 - $P = (1 + Q)' = 1' = 0$
 - $Q = (1 + P)' = 1' = 0$

$$P = (S + Q)'$$
$$Q = (R + P)'$$

Figure 6.5 A NOR gate latch.



SR-Latch Operation

As a starting point, assume that

- *set* input S is 1 and *reset* input R is 0
- At the next point
 - *set* input S is returned to 0
 - *reset* input R is first given the value 1 and then the value 0
- Finally, the latch is not operated with both S and R active, since, if $S = 1$ and $R = 1$, then both $P = 0$ and $Q = 0$
 - *indeterminate* because P and Q are complement of each other

SR-Latch Truth Table

Truth table for *SR*-latch is as follows:

<i>S</i>	<i>R</i>	<i>Q</i>	<i>Q'</i> (or <i>P</i>)	state
1	0	1	0	<i>Set</i>
0	0	1	0	<i>Hold</i>
0	1	0	1	<i>Reset (Clear)</i>
0	0	0	1	<i>Hold</i>
1	1	0	0	<i>Indeterminate</i>

Clocked RS-Latch

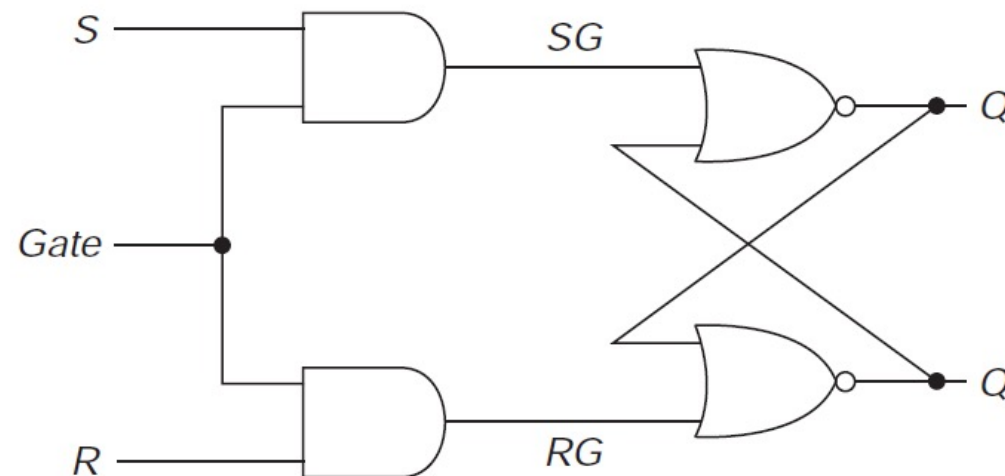
- In the previous *RS-latch*, the cross coupled connection from the output of one gate to the input of other gate constitutes a *feedback* path
- For this reason, these circuits are classified as *Asynchronous Sequential Circuits*
- By adding gates to the inputs of the basic circuit, the latch can be made to respond to input levels during the occurrence of a *clock pulse*

Gated Latch

We will look at a gated latch, as shown in Figure 6.6

- When *Gate* signal is *inactive* ($= 0$), *SG* and *RG* are both 0, and the latch remains *unchanged*
- Only when *Gate* goes to 1, can a 0 or 1 be stored in the latch

Figure 6.6 A gated latch.



Behavioral Table and Equation

- Behavioral table for gated latch, summarizing its operation is shown
- Behavioral equation can be obtained from K-map as: $Q(t+1) = (S + Q)R'$

Q	S	R	$Q(t+1)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	Indeterminate
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	Indeterminate

Flip Flops

- Flip flop is a clocked binary storage device, that is, a device that stores either a 0 or a 1
- Under normal operation, that value will only change on the appropriate transition of the clock
- State of system (i.e., what is in memory) changes on transition of clock
- Edge Triggered Flip Flop

Edge Triggered Flip Flop

- For some flip flops, that change takes place when clock goes from 1 to 0
– referred to as *trailing-edge triggered*
- For others, that change takes place when clock goes from 0 to 1
– referred to as *leading-edge triggered*



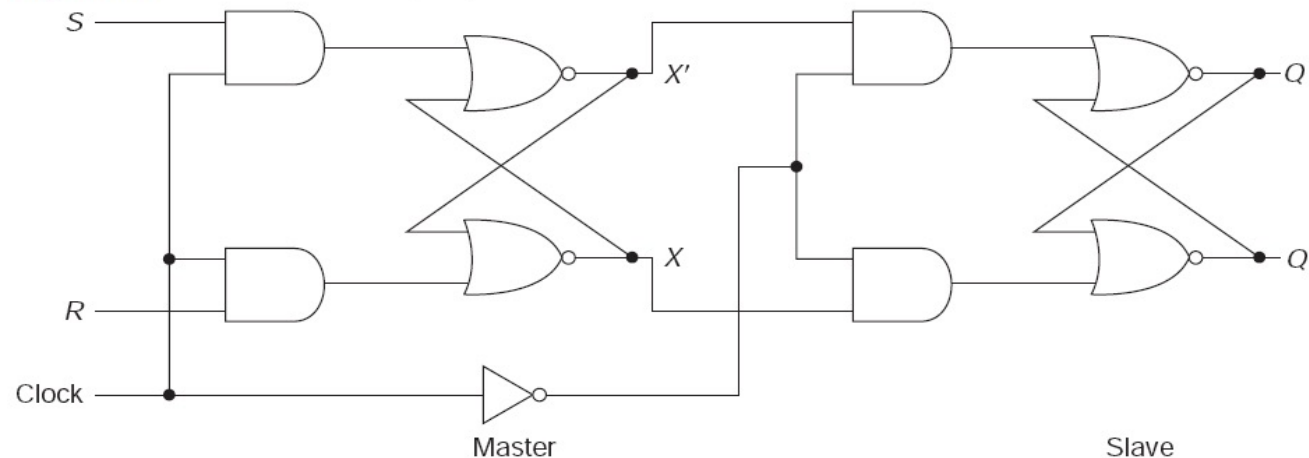
Flip Flop Output

- What is stored after the transition depends on
 - flip flop data inputs and
 - what was stored in the flip flop prior to the transition
- Flip flops have one or two outputs
 - One output is the **state** of the flip flop
 - If there are two outputs, the other output is the **complement of state**

Master/Slave Flip Flop

- Simple **SR master/slave** FF can be constructed with two gated latches
- When **clock** is **1**, the **S** and **R** inputs establish the values for the first FF, **master** (enabled); during that time, **slave** is not enabled
- As soon as **clock** goes to **0**, **master** is **disabled** and **slave** is **enabled**

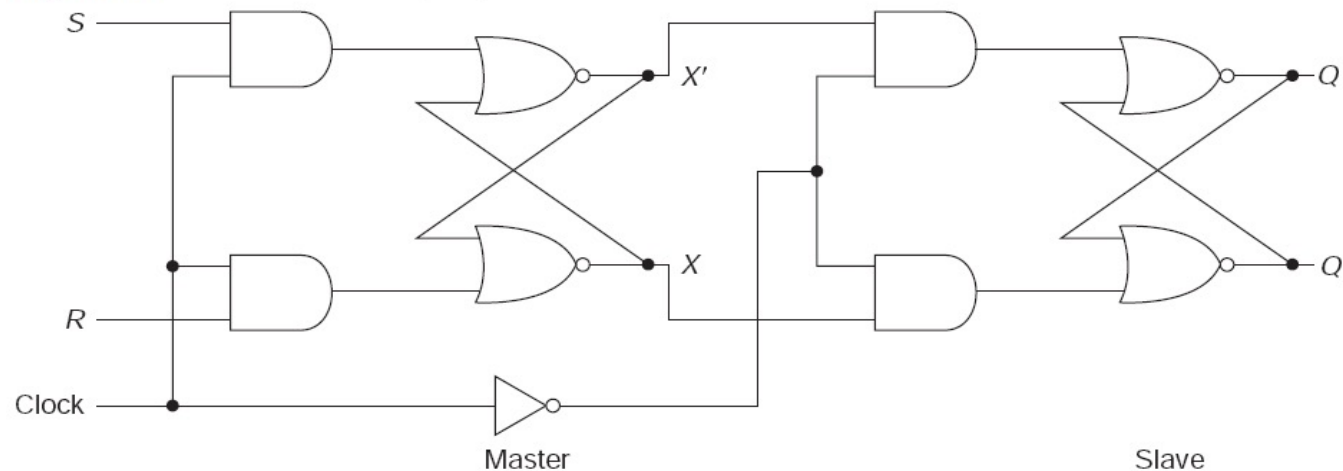
Figure 6.7 A master/slave flip flop.



Master/Slave Flip Flop

- Values of the master's outputs, X and X' , are determined by the value of S and R just before the trailing edge (when clock goes from 1 to 0)
 - These are the inputs of the slave
- Slave (and FF output) changes as clock goes from 1 to 0 (on the trailing edge) and remains that way until the next clock cycle

Figure 6.7 A master/slave flip flop.



Master/Slave Flip Flop

Note:

- We could get a *leading-edge triggered* flip flop by connecting the clock to the slave and its complement to the master
- The change in the output of the flip flop is delayed from the edge of the clock (a somewhat longer delay than that through a gate)

Flip Flop Types

Commonly used flip flops are, *D* and *JK*

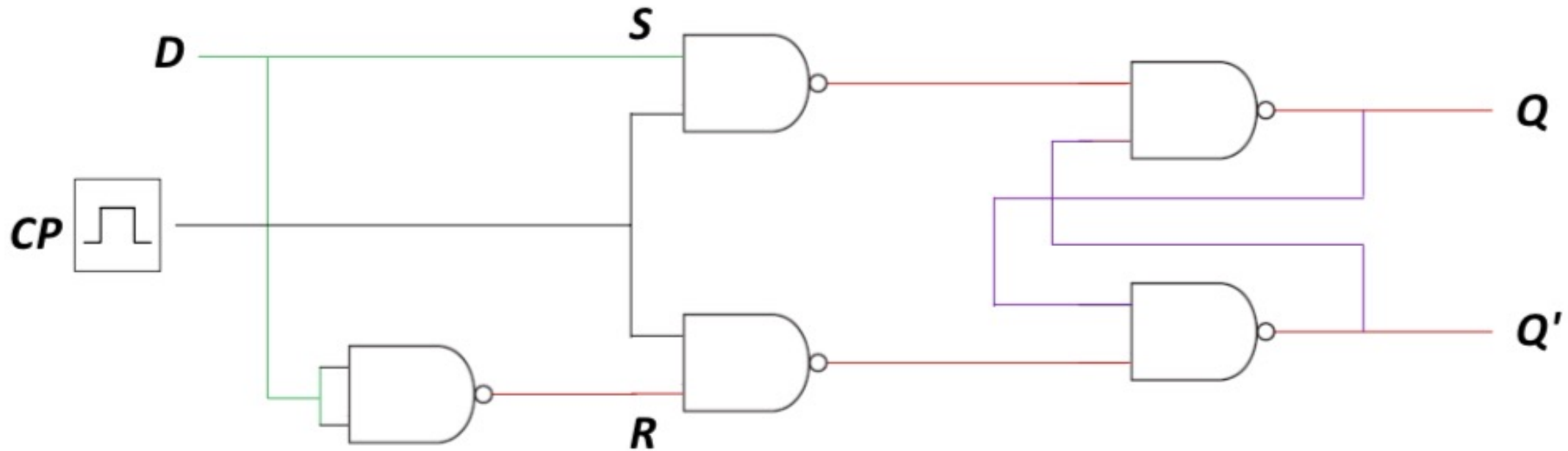
- *D* flip flop is the most straightforward and is commonly found in programmable logic devices
- *JK* flip flop almost always produces the simplest combinational logic
- We will also introduce the *SR* and *T* flip flops since they naturally lead to the *JK* flip flop

D-Flip Flop

- The simplest flip flop is the *D* flip flop
- Name comes from *Delay*, since the output is just the input delayed until the next active clock transition
- Next state of *D* flip flop is the value of *D* before the clock transition

D-Flip Flop

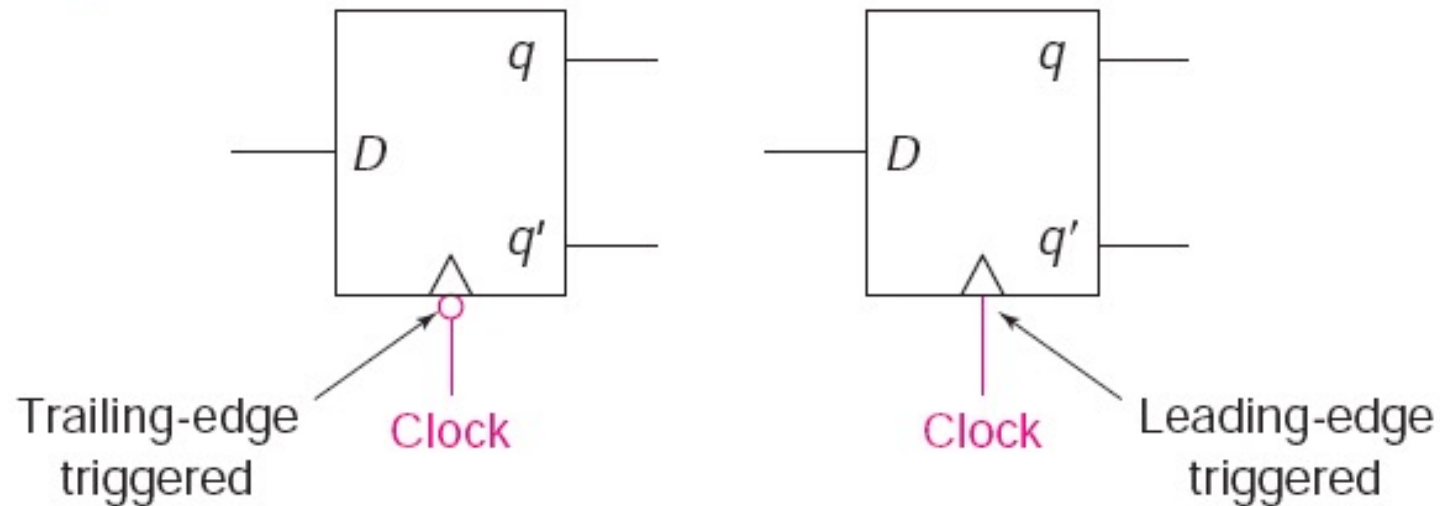
D-flip flop is shown below (with NAND gates) is the modification of clocked *RS*-flip flop



Block Diagram

- Block diagrams of D flip flops are shown in Figure 6.8, both
 - trailing-edge triggered
 - leading-edge triggered

Figure 6.8 D flip flop diagrams.



Behavioral Table

- Behavioral table for D -flip flop is shown below
- The table on the right describes the same information, but it shows only the flip flop input(s) in input column(s) and next state in output column

Table 6.3 The D flip flop behavioral tables.

D	q	$q^{\star}$
0	0	0
0	1	0
1	0	1
1	1	1

D	$q^{\star}$
0	0
1	1

Behavioral Equation

- From this truth table, we can conclude that

$$q\star = D$$

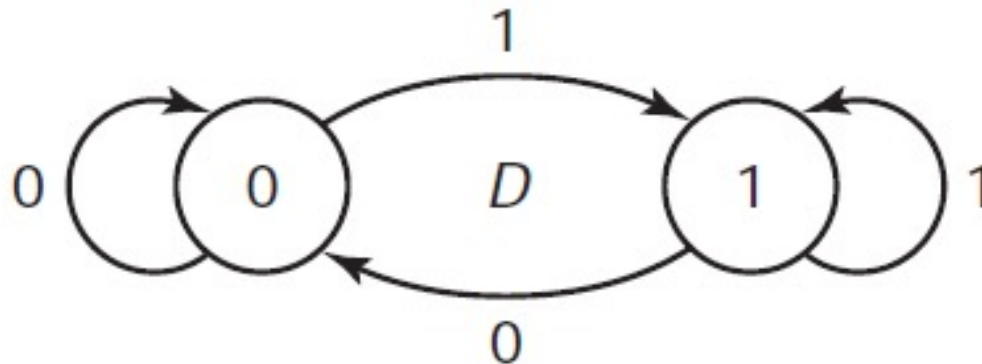
which is the behavioral equation of D-flip flop

- We can also get the same equation by drawing K-map for $q\star$

State Diagram

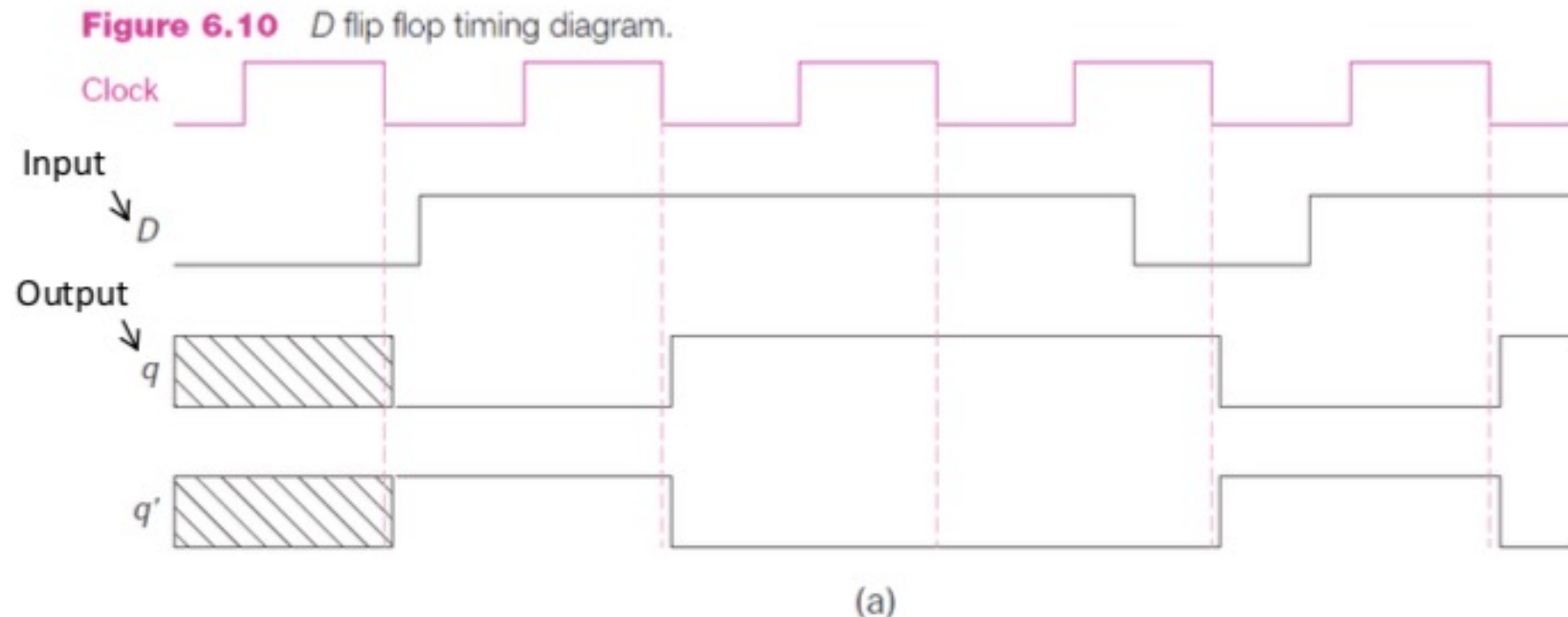
- State diagram for a D flip flop is shown
 - It has two states (for all types of flip flops)
- Transition paths are labeled with input (i.e. D) that causes that transition
- Output is not shown in circle since it is same as the state

Figure 6.9 D flip flop state diagram.



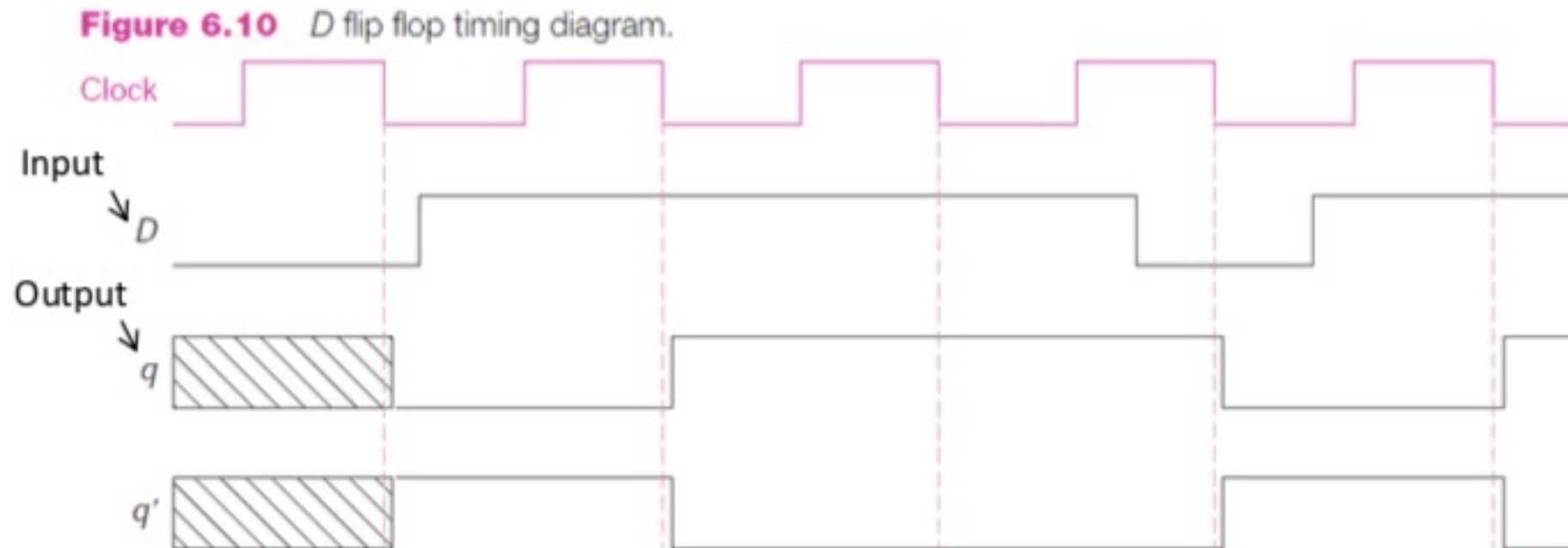
Timing Diagram

- Behavior of a trailing-edge triggered D FF is illustrated in timing diagram
- It should be kept in mind that behavior of flip flop at a clock transition depends on the value of the flip flop inputs prior to that transition



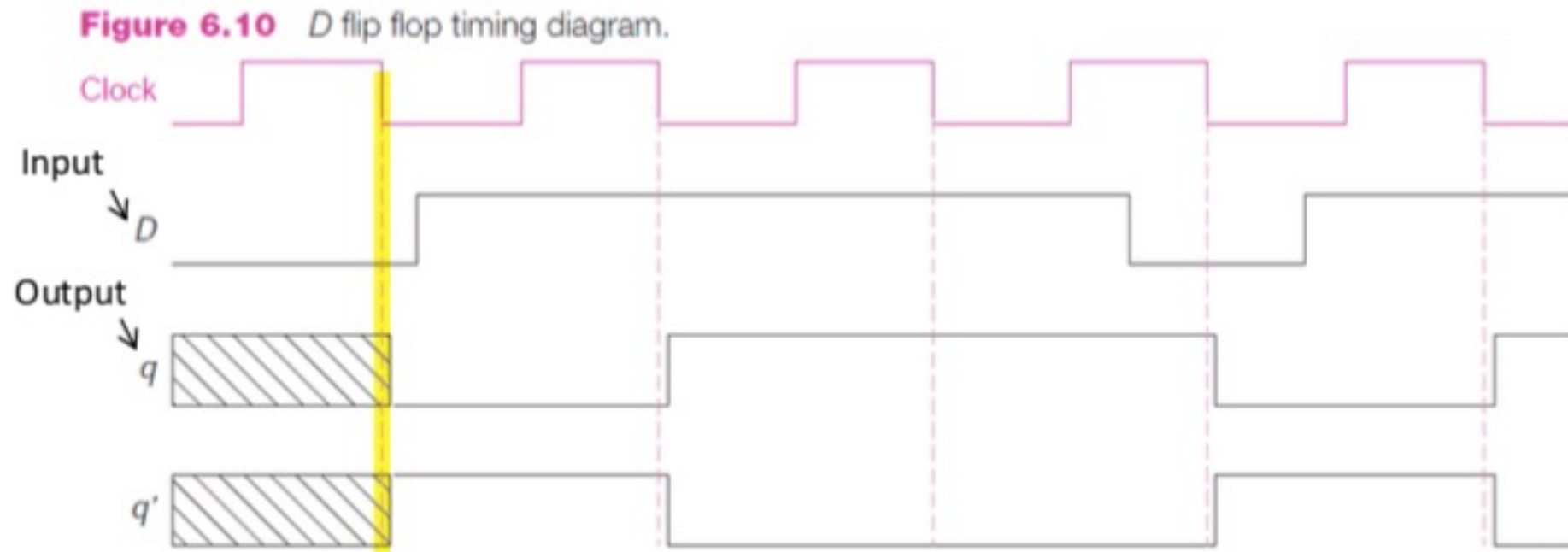
Trailing Edge Triggered *D*-Flip Flop

Unless we know initial value of q , i.e., what was stored in FF before we started to look, then q is **unknown** until after the negative-going clock transition; indicated by the slashed section on the timing diagram



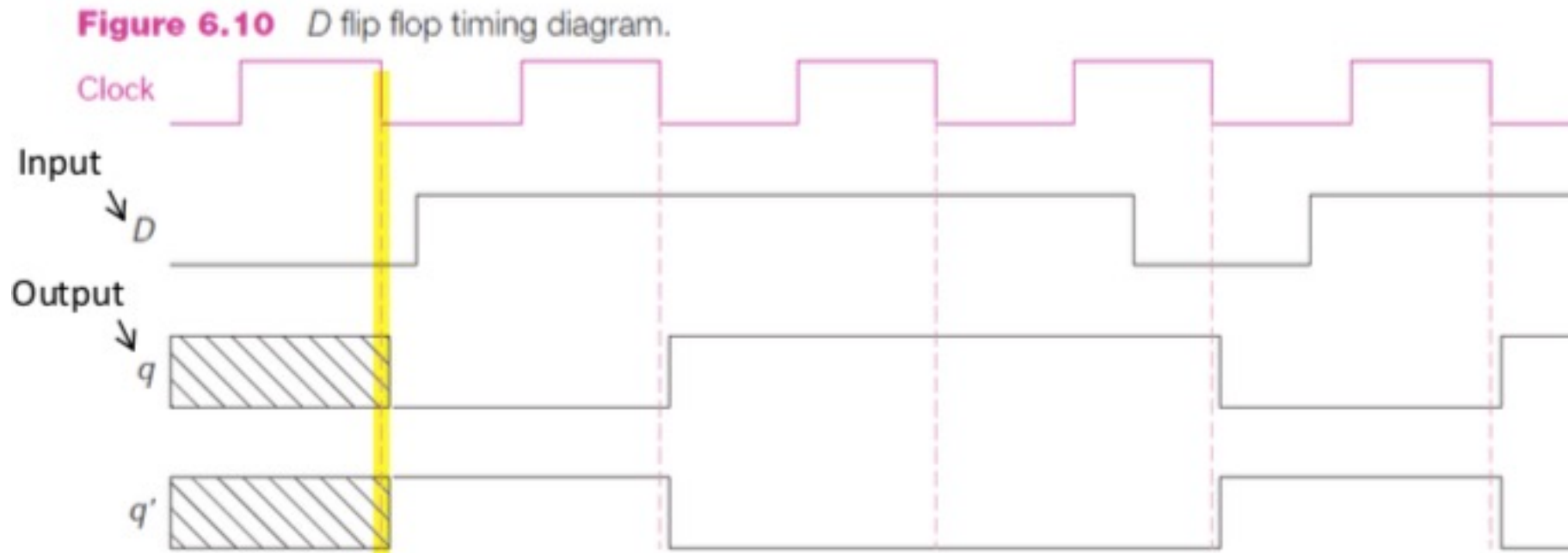
Trailing Edge Triggered *D*-Flip Flop

- When the first trailing edge (going from 1 to 0) of the clock occurs, the state of the flip flop is established
- Since *D* is 0 at that time, *q* goes to 0 (*q'* goes to 1)



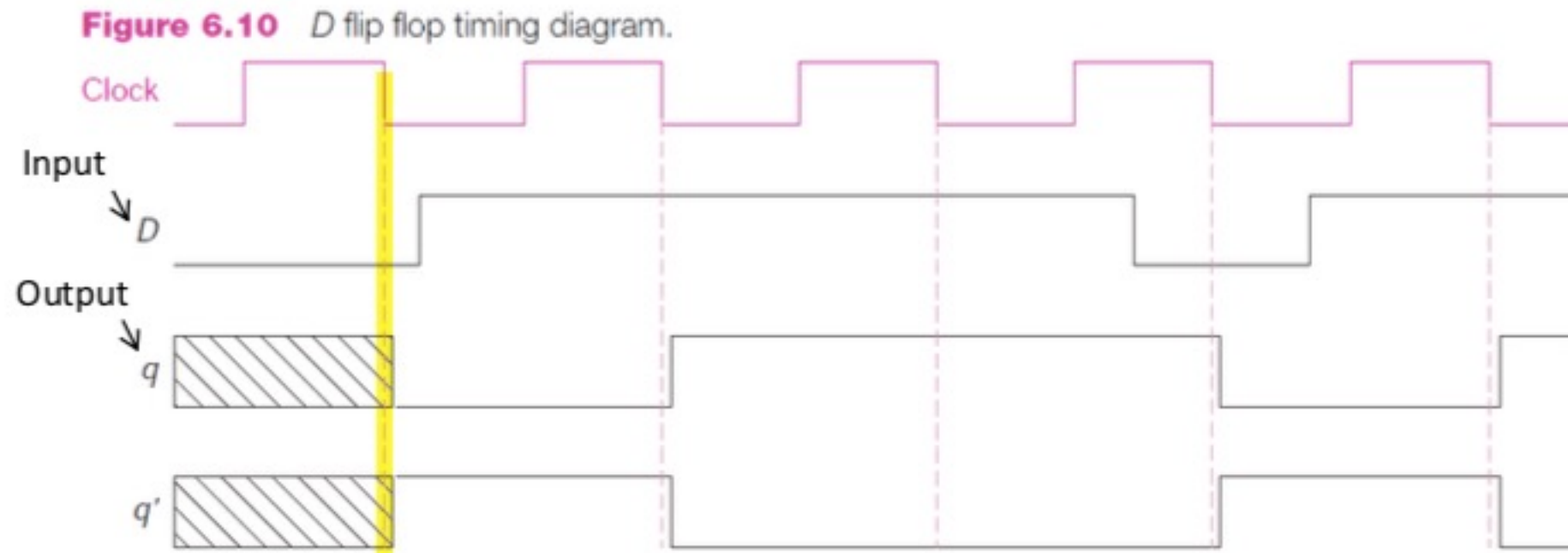
Trailing Edge Triggered *D*-Flip Flop

- Note that there is a slight **delay** in the output
- Input, *D*, usually changes shortly after transition, as shown for the first change, but may change at any time, as long as it has reached the correct value well before the next active transition



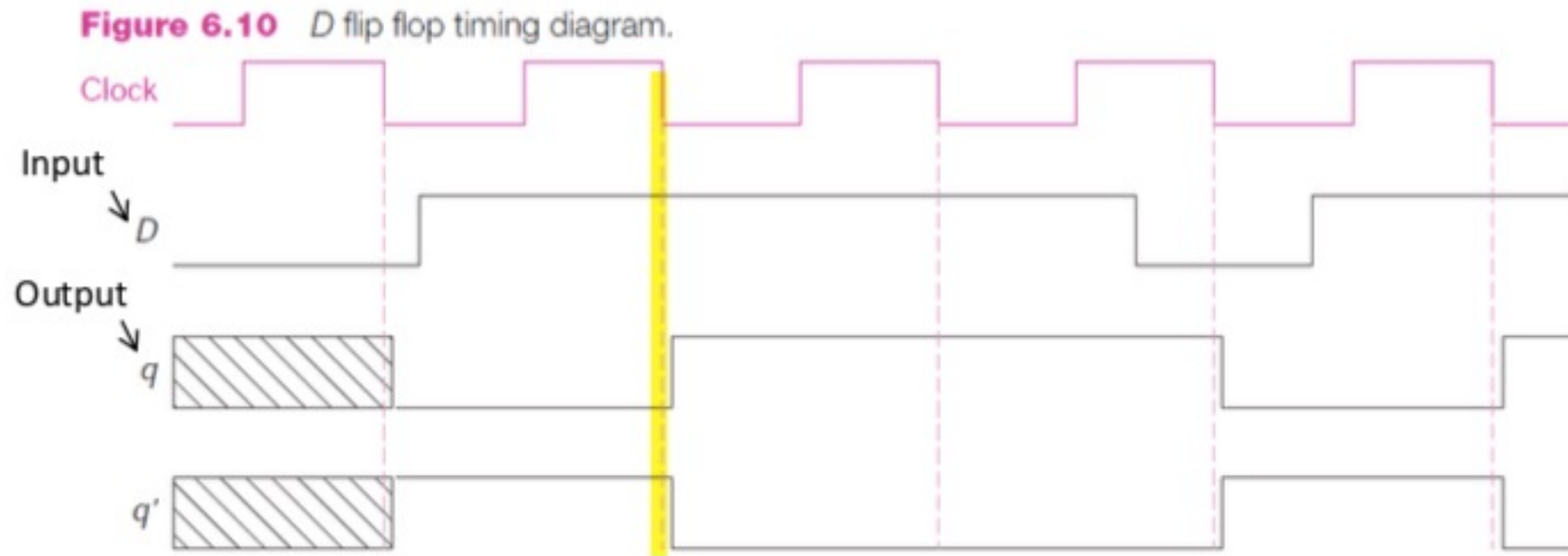
Trailing Edge Triggered D -Flip Flop

- Second and third changes in D come later in clock cycle
- As shown, q' output is opposite of q output



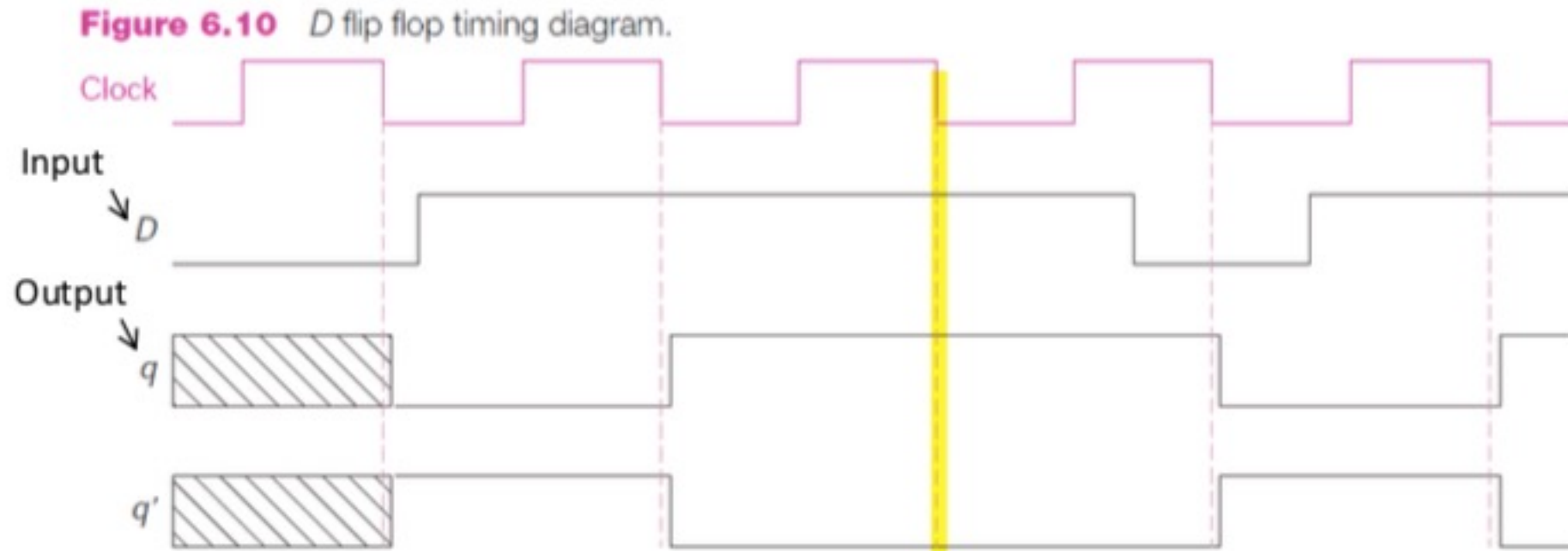
Trailing Edge Triggered *D*-Flip Flop

At second trailing edge, *D* is 1
– *q* is 1 for the next clock period



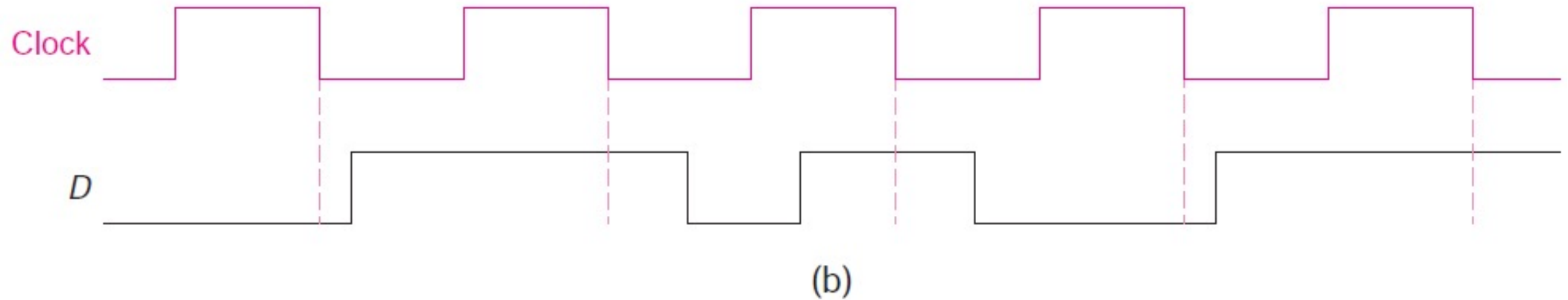
Trailing Edge Triggered *D*-Flip Flop

At third trailing edge, *D* is still 1
– *q* remains 1 for another clock period



Trailing Edge Triggered *D*-Flip Flop

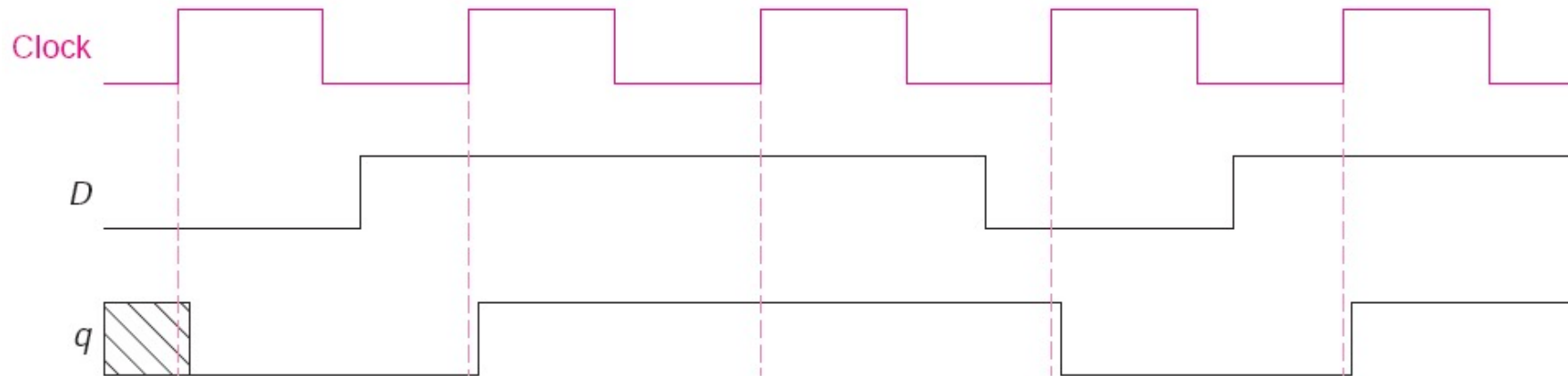
If *D* input were to go back and forth between clock transitions, as shown in Figure 6.10b, output would not be affected, since value of *D* is only relevant near time of a trailing edge; it would be same as in Figure 6.10a



Leading Edge Triggered D -Flip Flop

Timing diagram for a leading-edge triggered D flip flop, using the same input as before, is shown in Figure 6.11

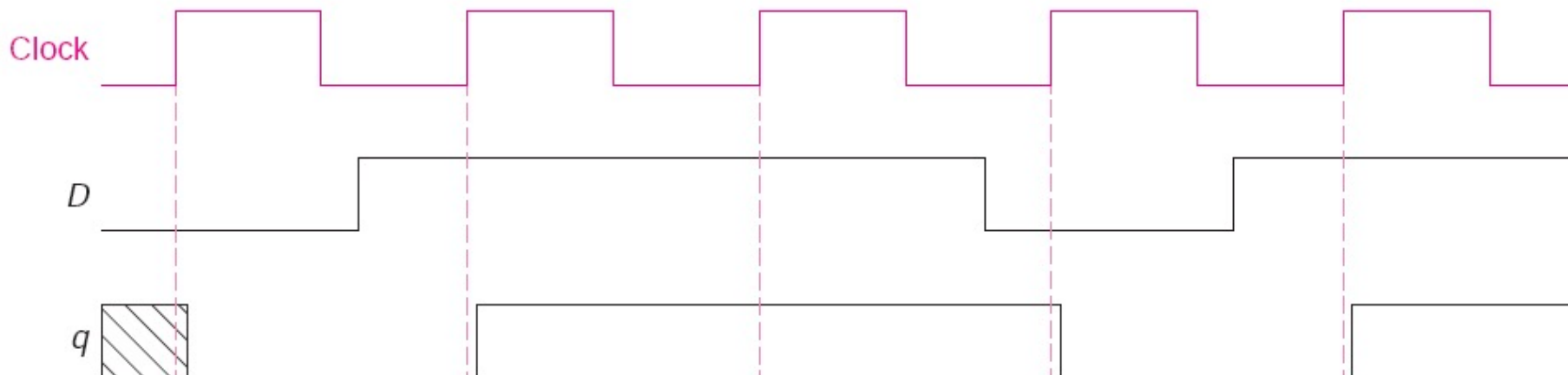
Figure 6.11 Leading-edge triggered D flip flop.



Leading Edge Triggered *D*-Flip Flop

- Tables describing the flip flop need not be modified; the only difference is when the output changes relative to the clock
- Output (state of flip flop) changes shortly after clock goes from 0 to 1 (based on the input just before that transition)

Figure 6.11 Leading-edge triggered *D* flip flop.



Flip flops with *Clear* and *Preset* Inputs

- *SR* Flip Flop
- *JK* Flip Flop
- *T* Flip Flop

Flip flops with *Clear* and *Preset* Inputs

- Before going on to look at other types of flip flops, we will examine the behavior of flip flops with static (asynchronous)
 - *Clear* (or *Reset*) and
 - *Preset* (or *Set*) inputs
- Any type of flip flop may have one or both of these available

Flip flops with *Clear* and *Preset* Inputs

- *D*-flip flop with **active low clear** and *preset* inputs is shown
- Behavior of this flip flop is described by the truth table

Figure 6.14 Flip flop with clear and preset inputs.

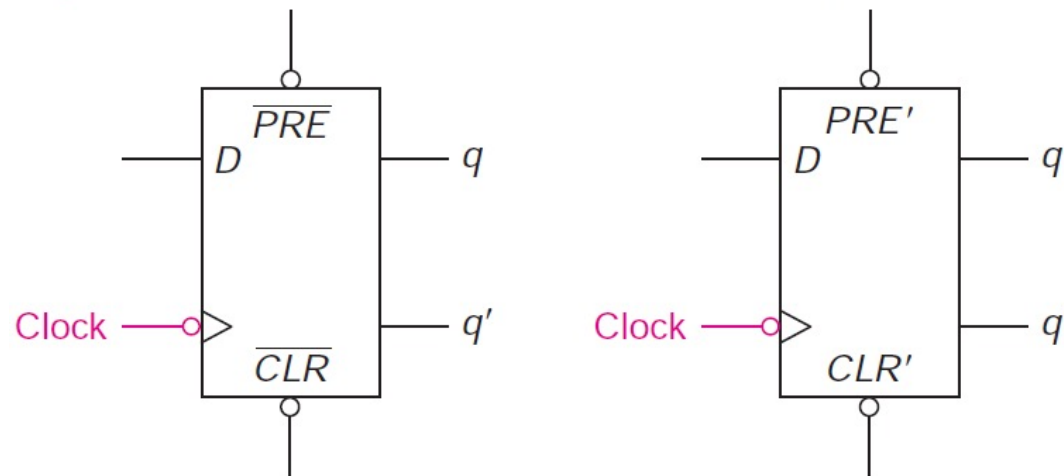


Table 6.4 *D* flip flop with clear and preset inputs behavioral table.

PRE'	CLR'	D	q	q^*
0	1	X	X	1 static
1	0	X	X	0 immediate
0	0	X	X	— not allowed
1	1	0	0	0
1	1	0	1	0 clocked
1	1	1	0	1 (as before)
1	1	1	1	1

Clear and Preset Inputs

Clear and *Preset* inputs

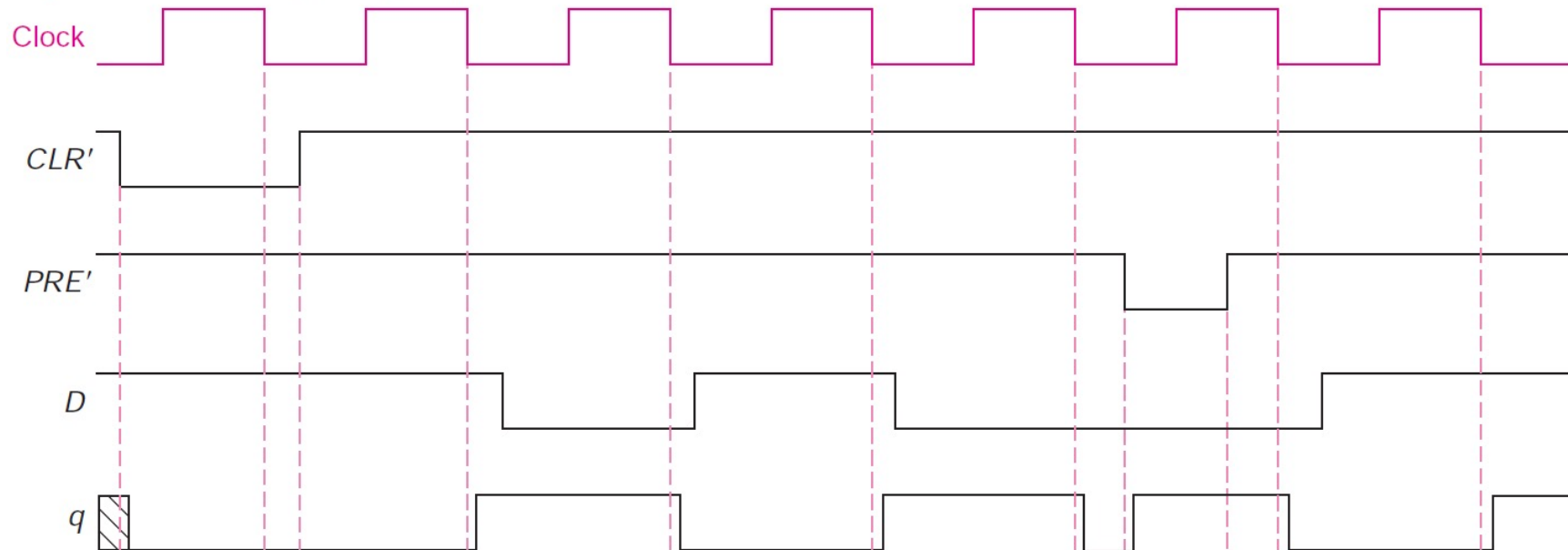
- act immediately (except for circuit delay)
 - override the clock, that is, they force output to 0 and 1, respectively
- Only when both of these static inputs are 1
 - Flip flop behave as before
 - Clock transition and *D* input determining behavior

Timing Diagram

A timing example is shown in Figure 6.15

- *clear* input becomes active near beginning of time shown, forcing q to 0

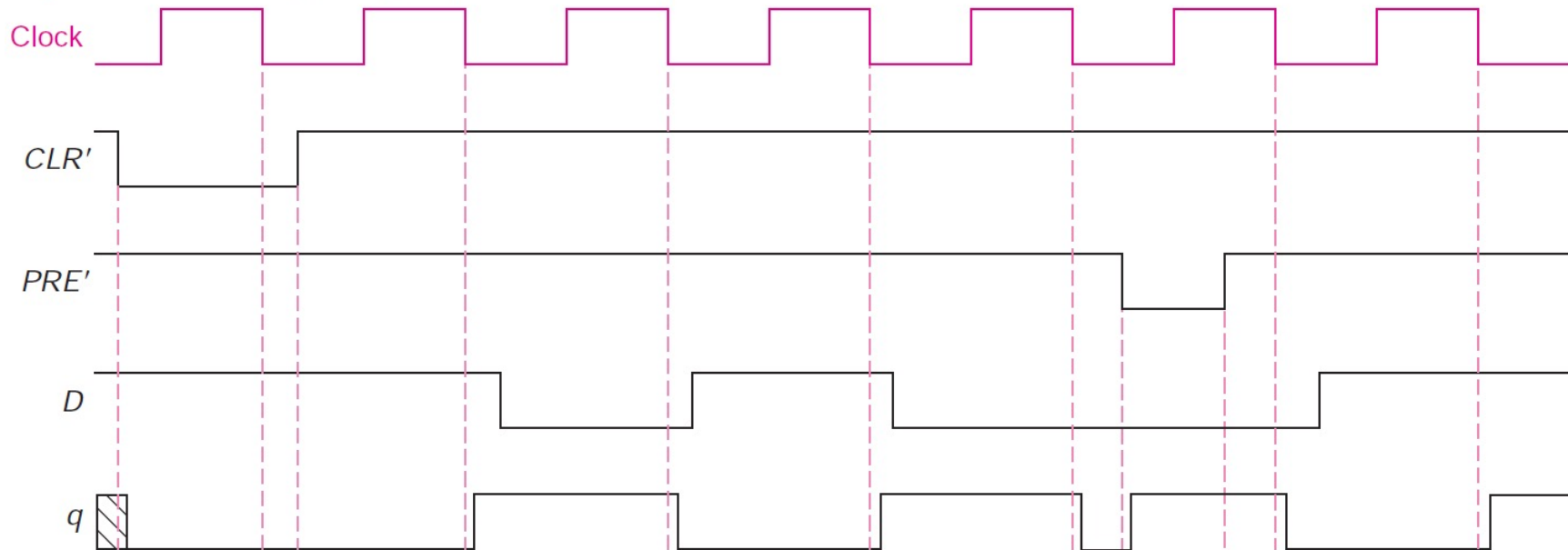
Figure 6.15 Timing for flip flop with clear and preset.



Timing Diagram

As long as that input remains 0, clock and D are ignored; thus, nothing changes at first trailing edge (going from 1 to 0) of clock that is shown

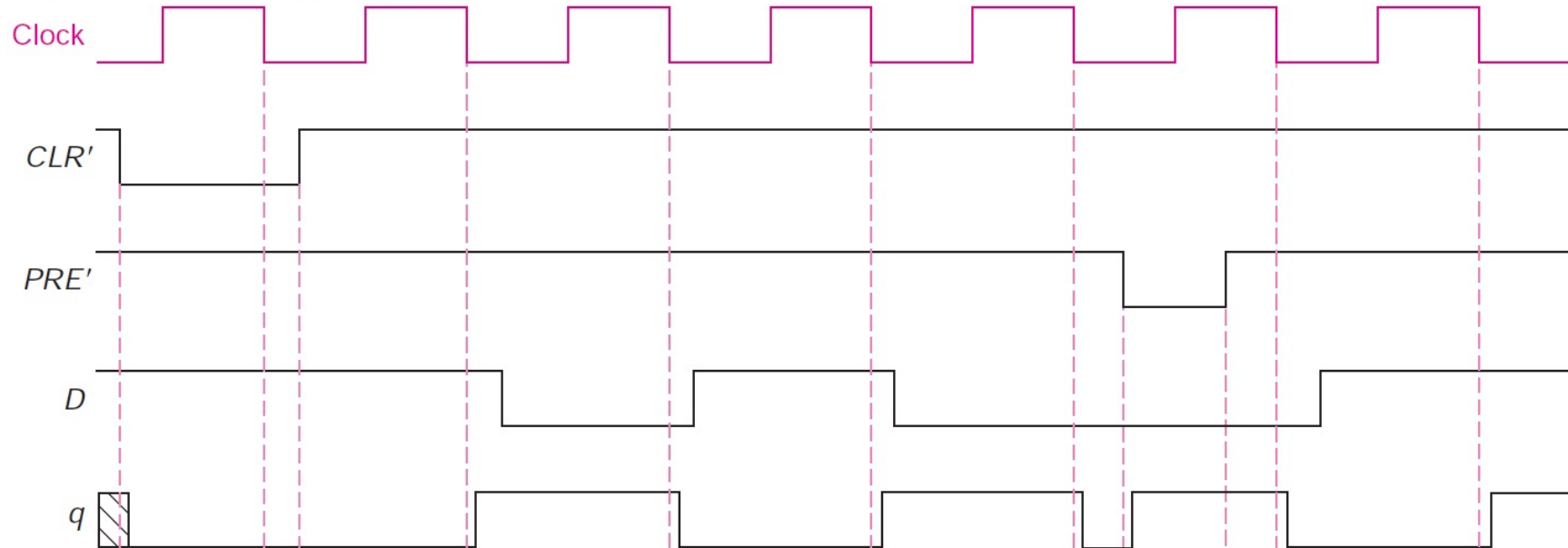
Figure 6.15 Timing for flip flop with clear and preset.



Timing Diagram

- Once the *clear* returns to 1, then *clock* and *D* take over; but they have no effect until the next trailing edge of the clock
- *D* input determines behavior of flip flop at next four trailing edges

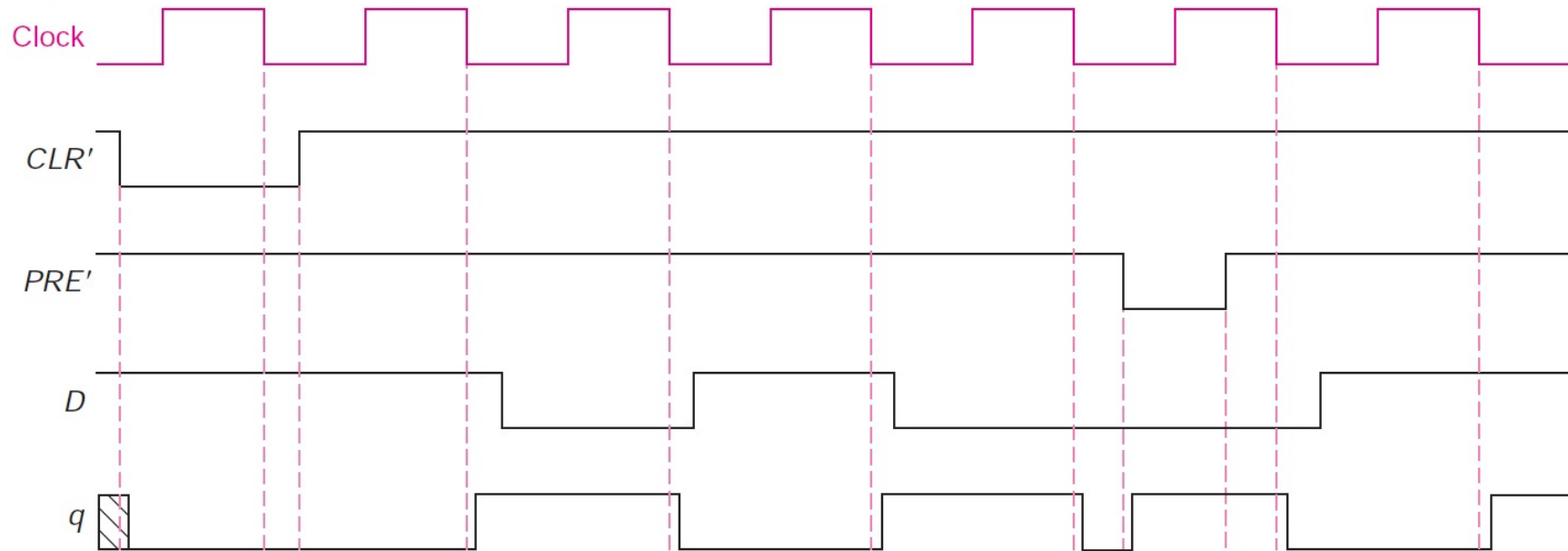
Figure 6.15 Timing for flip flop with clear and preset.



Timing Diagram

- When *preset* input goes to 0, flip flop output goes to 1
- When *preset* input goes back to 1, clock and *D* once again take over

Figure 6.15 Timing for flip flop with clear and preset.



SR-Flip Flop

SR (*Set-Reset*) flip flop has two inputs, *S* & *R*, same meaning as *SR* latch

- Behavioral tables when *clock pulse* is 1
- Behavioral equation obtained from K-map
- State diagram obtained from behavioral table

Table 6.5 *SR* flip flop behavioral tables.

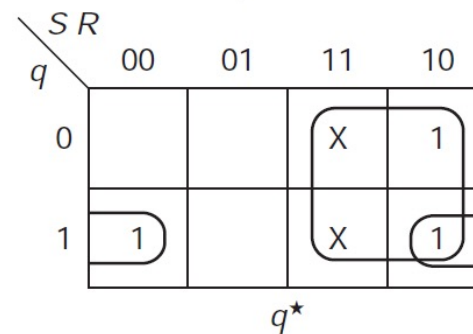
<i>S</i>	<i>R</i>	<i>q</i>	<i>q</i> [★]
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	—
1	1	1	—

not
allowed

<i>S</i>	<i>R</i>	<i>q</i> [★]
0	0	<i>q</i>
0	1	0
1	0	1
1	1	—

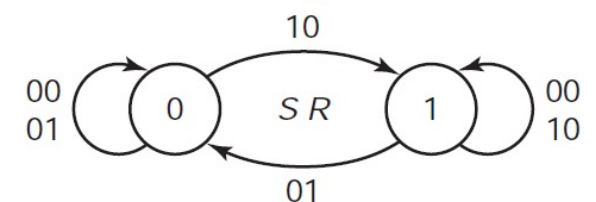
not allowed

Map 6.1 *SR* flip flop behavioral map.



$$q^{\star} = S + R'q$$

Figure 6.16 *SR* flip flop state diagram.



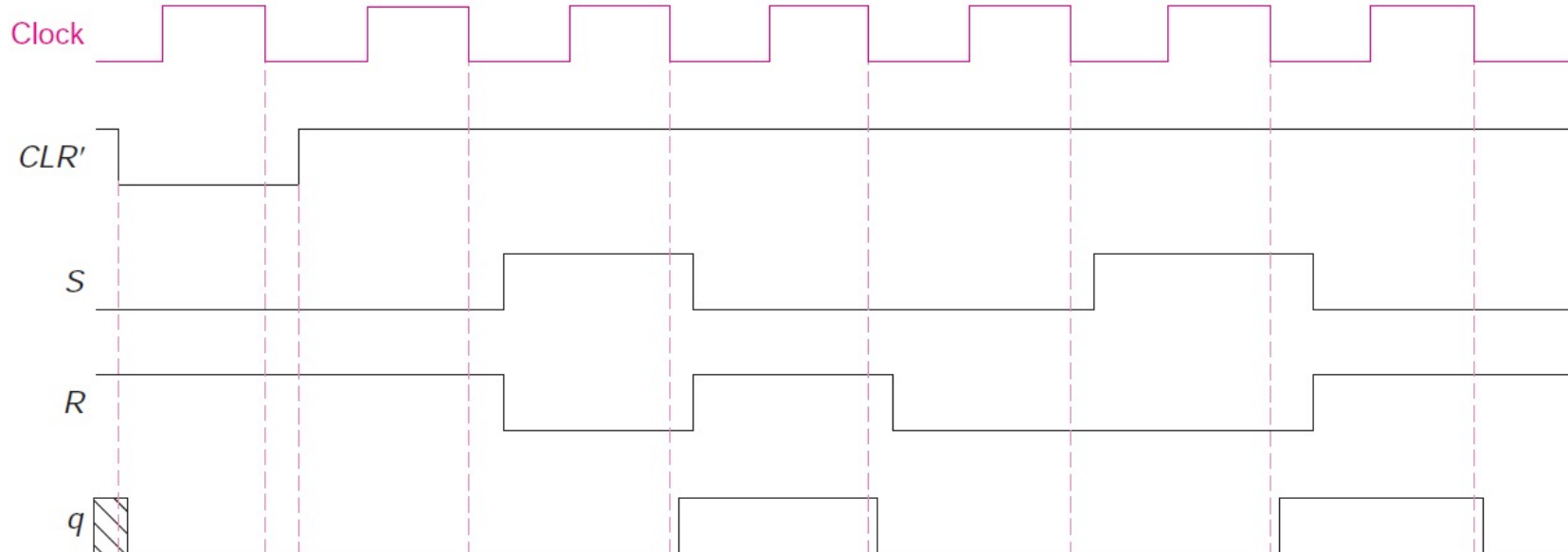
SR-Flip Flop

- *Set* (*S*) input causes a 1 to be stored in the flip flop at the next active clock edge
- Reset (*R*) input causes a 0 to be stored
- *S* and *R* inputs are never made 1 at the same time

Timing Diagram

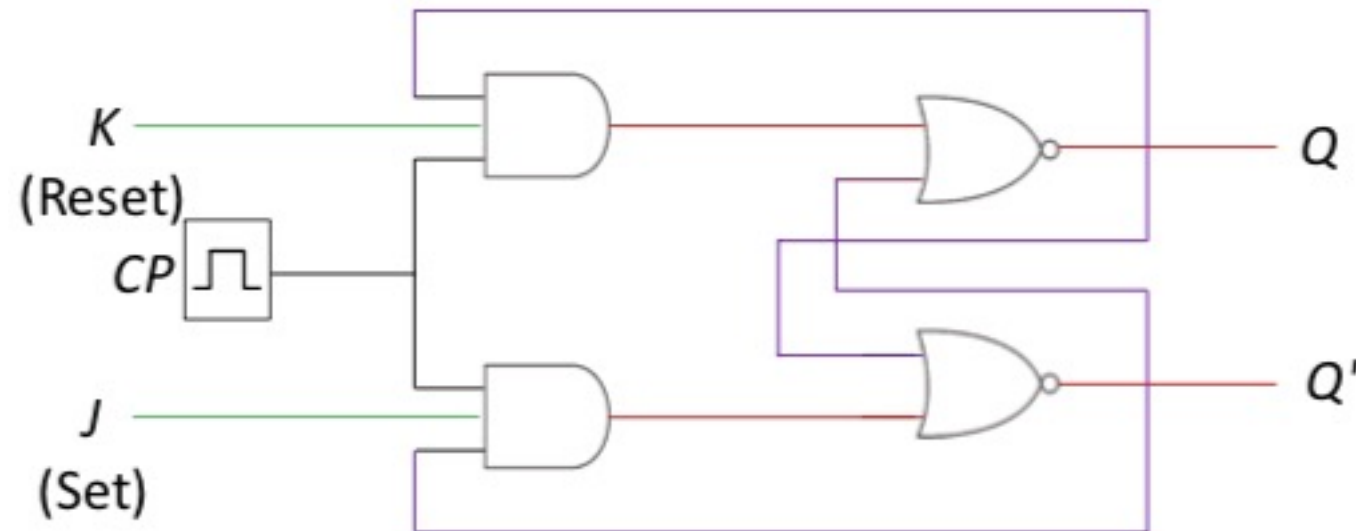
- Timing example (there is only *clear* input, not *preset* one) in Figure 6.17
- Note that we never made both *S* and *R* equal to 1 at the same time
 - Also, when both *S* and *R* are 0, *q* does not change

Figure 6.17 SR flip flop timing diagram.



JK-Flip Flop

- *JK*-flip flop is refinement of *RS* flip flop
 - *indeterminate* state of *SR* type is defined in the *JK* type
- Inputs *J* and *K* behave like *S* and *R* to *Set (Preset)* and *Clear (Reset)* the flip flop respectively
- Clocked *JK* flip flop is shown below



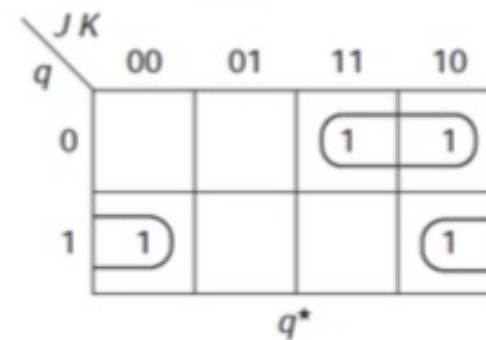
JK-Flip Flop

- Behavioral tables
- Behavioral equation obtained from K-map
- State diagram obtained from the behavioral table

Table 6.7 JK flip flop behavioral tables.

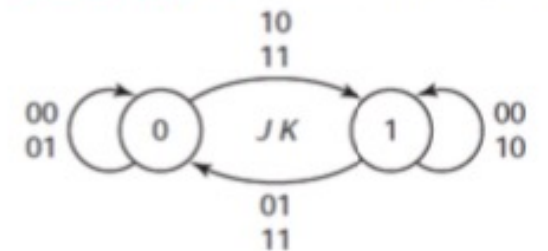
J	K	q	q^*
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Map 6.2 JK flip flop behavioral map.



$$q^* = Jq' + K'q$$

Figure 6.20 JK flip flop state diagram.

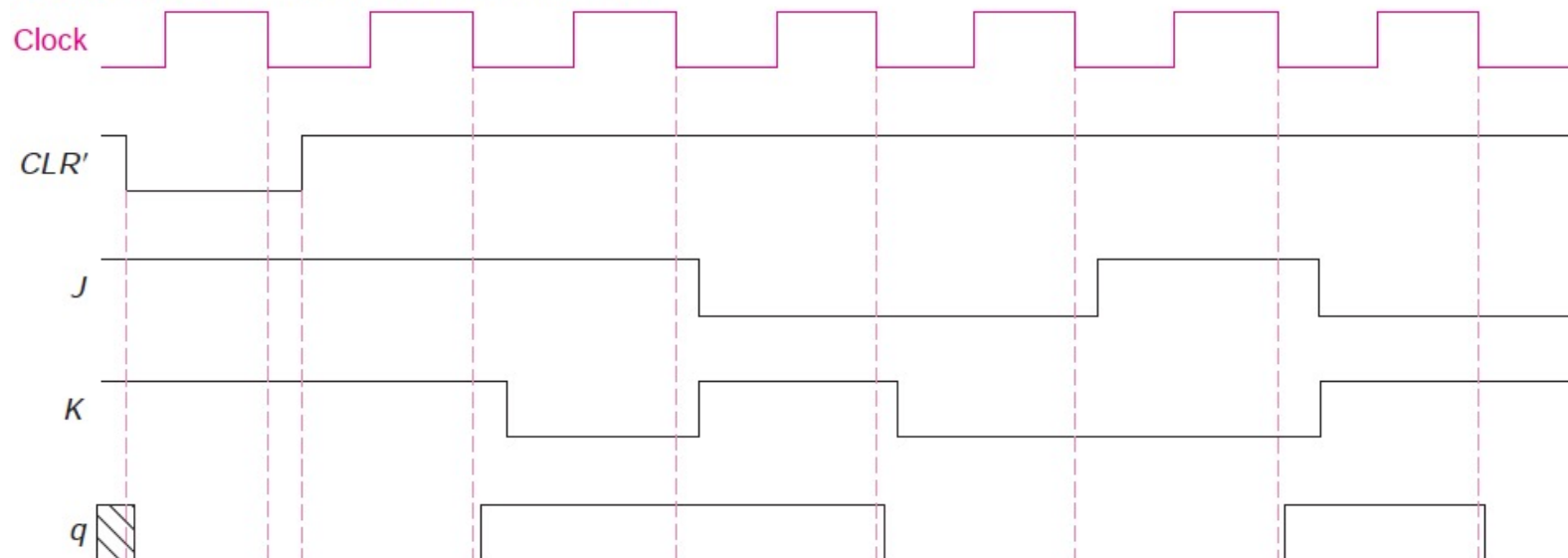


Timing Diagram

Timing example for JK flip flop is shown

- There are times when both J and K are 1 simultaneously; FF just changes state at those times, if $q = 1$ then it switches to $q = 0$ and vice versa
- Also evident from the behavioral table

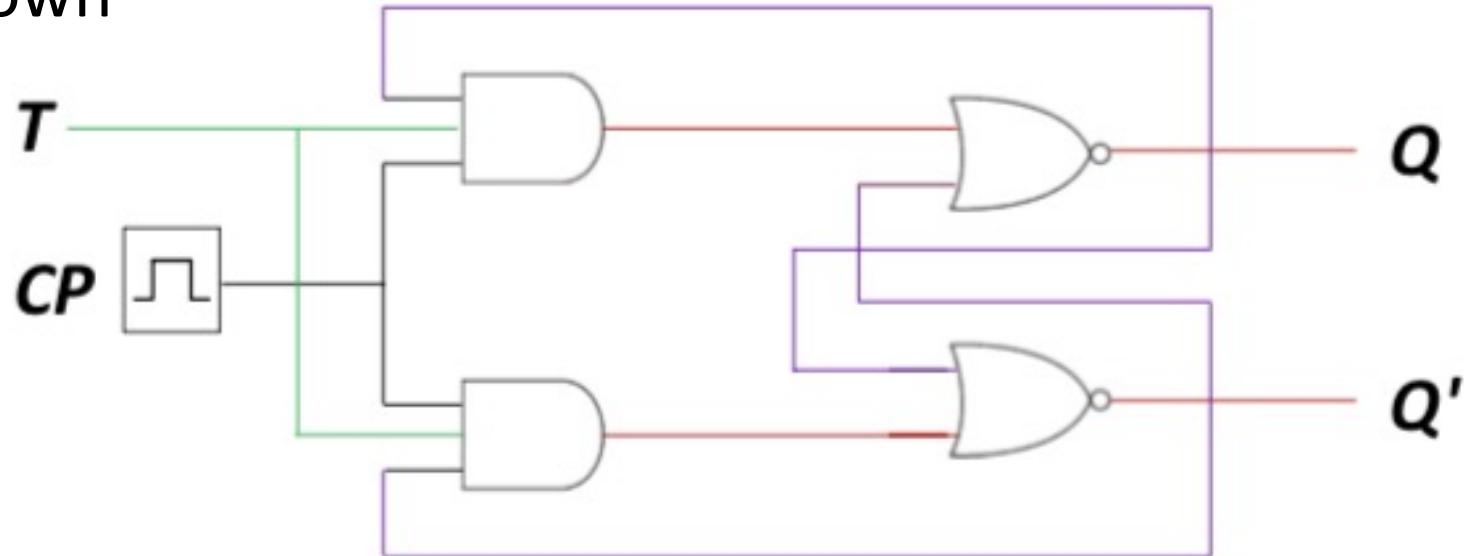
Figure 6.21 Timing diagram for JK flip flop.



T-Flip Flop

A T (i.e. *toggle*)-flip flop is a single input version of the JK -flip flop

- It has one input, T , such that
 - if $T = 1$, the flip flop changes state (that is, is toggled), and
 - if $T = 0$, the state remains the same
- A *clocked* T-flip flop is shown



Behavioral Table

- Behavioral tables
- Behavioral equation
- State diagram obtained from the behavioral table

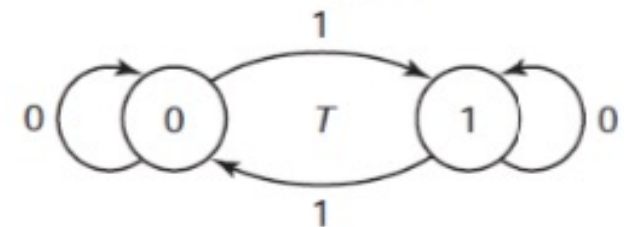
Table 6.6 T flip flop behavioral tables.

T	q	q^*
0	0	0
0	1	1
1	0	1
1	1	0

T	q^*
0	q
1	q'

$$q^* = T \oplus q$$

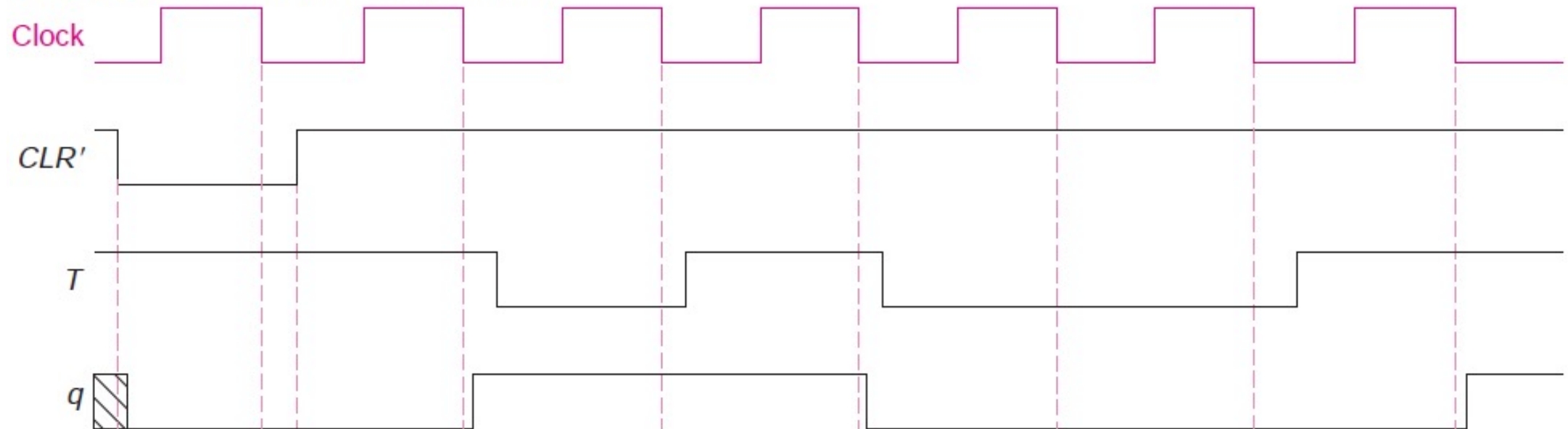
Figure 6.18 T flip flop state diagram.



Timing Diagram

- Timing example for the T -flip flop is shown

Figure 6.19 T flip flop timing diagram.



Analysis of Sequential Systems

Moore and Mealy

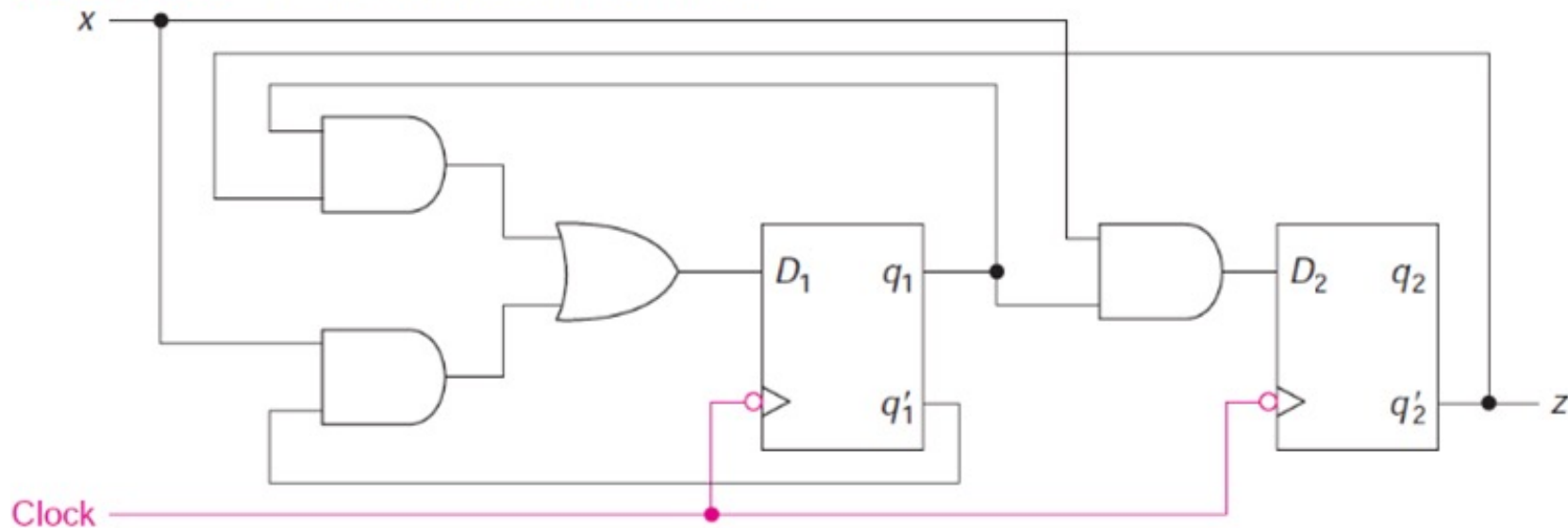
Analysis of Sequential Systems

- We are going to examine some small state machines (consisting of flip flops and gates) and analyze their behavior
 - Produce timing diagrams, timing traces, state tables, and state diagrams
- We will also look at relationship between the state table and the timing

Example: Moore Model

- Consider the following circuit with two trailing-edge (going from 1 to 0) triggered D flip flops
- Call flip flops q_1 and q_2 ; sometimes use names, such as A and B

Figure 6.22 A D flip flop Moore model circuit.

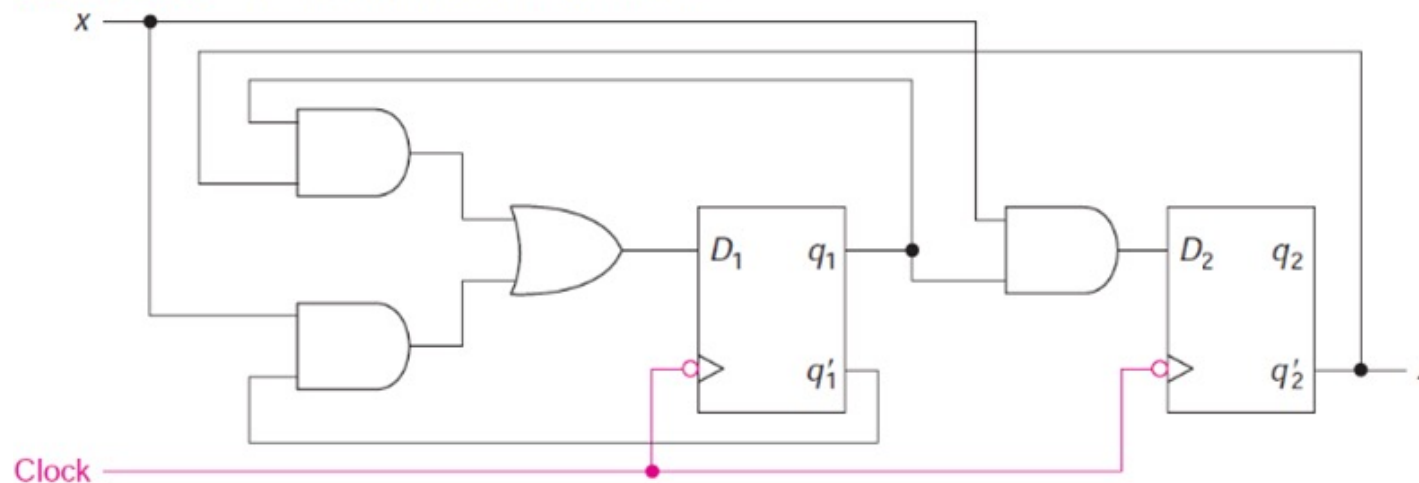


Example cont.

We have

- x is input to circuit
- D_1 is input to first flip flop and D_2 is input to second flip flop
- q_1 and q_2 are outputs of first and second flip flop respectively
- z is output of the entire circuit

Figure 6.22 A D flip flop Moore model circuit.



Example cont.

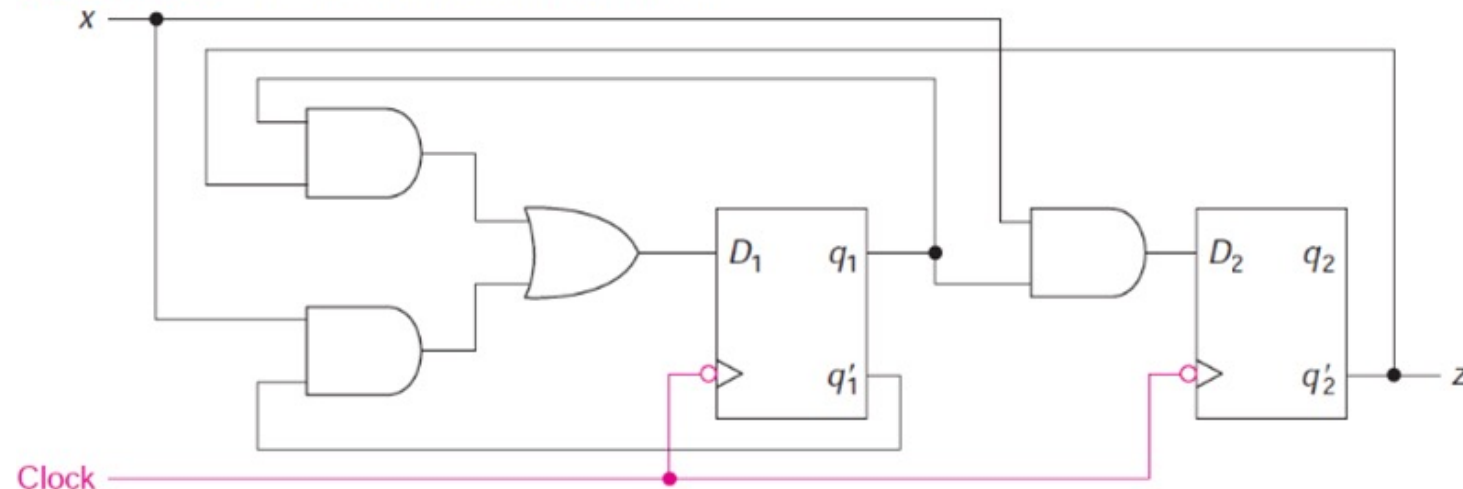
- Since output z , which equals q'_2 , is a function of state (contents of FFs)
- and not of the input x , therefore it is a **Moore model**
- From this circuit, we find

$$q1^{\star} = D1 = q1q'2 + xq'1$$

$$q2^{\star} = D2 = xq1$$

$$z = q'2$$

Figure 6.22 A D flip flop Moore model circuit.



Example cont.

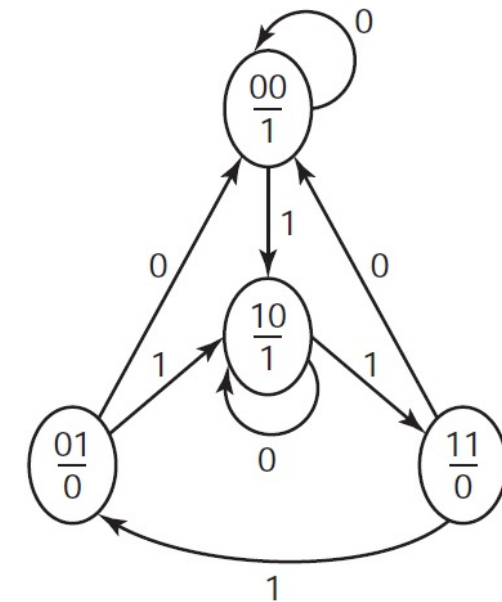
- Using the equations, we can construct the following state table
- The corresponding state diagram (i.e. Moore model) is shown below

$$q_1^* = D_1 = q_1 q_2' + x q_1'$$
$$q_2^* = D_2 = x q_1$$
$$z = q_2'$$

Table 6.8b Complete state table.

$q_1 q_2$	$q_1^* \quad q_2^*$		z
	$x = 0$	$x = 1$	
0 0	0 0	1 0	1
0 1	0 0	1 0	0
1 0	1 0	1 1	1
1 1	0 0	0 1	0

Figure 6.23 A Moore state diagram.

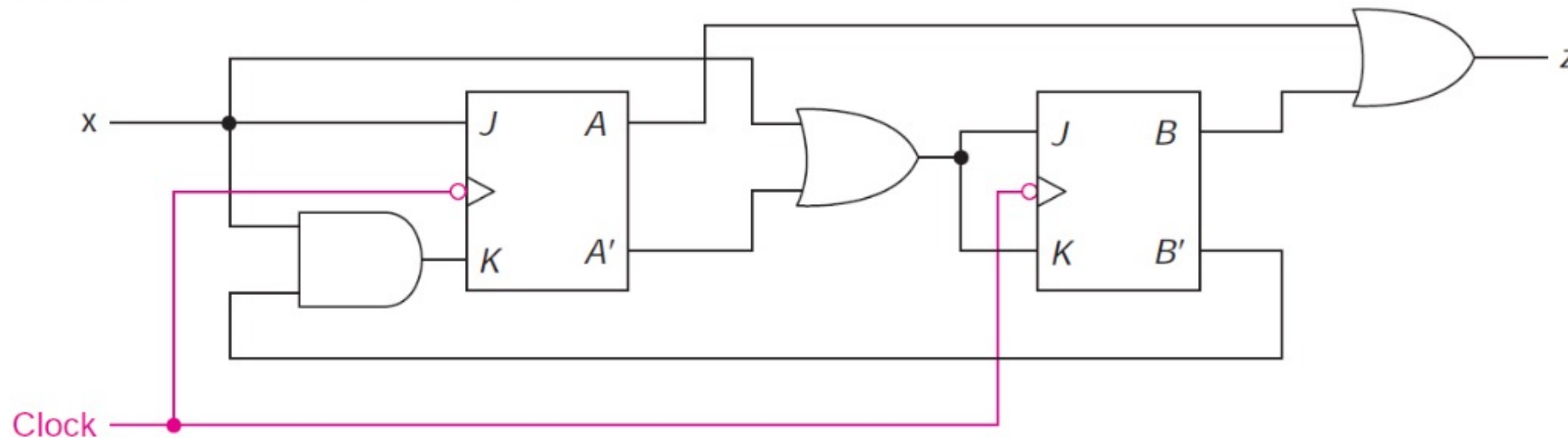


Example

Consider the following circuit with two JK flip flops

- Since output z , which equals $A+B$, is a function of the state and not of the input x , there fore it is a **Moore model**

Figure 6.24 A Moore model circuit.

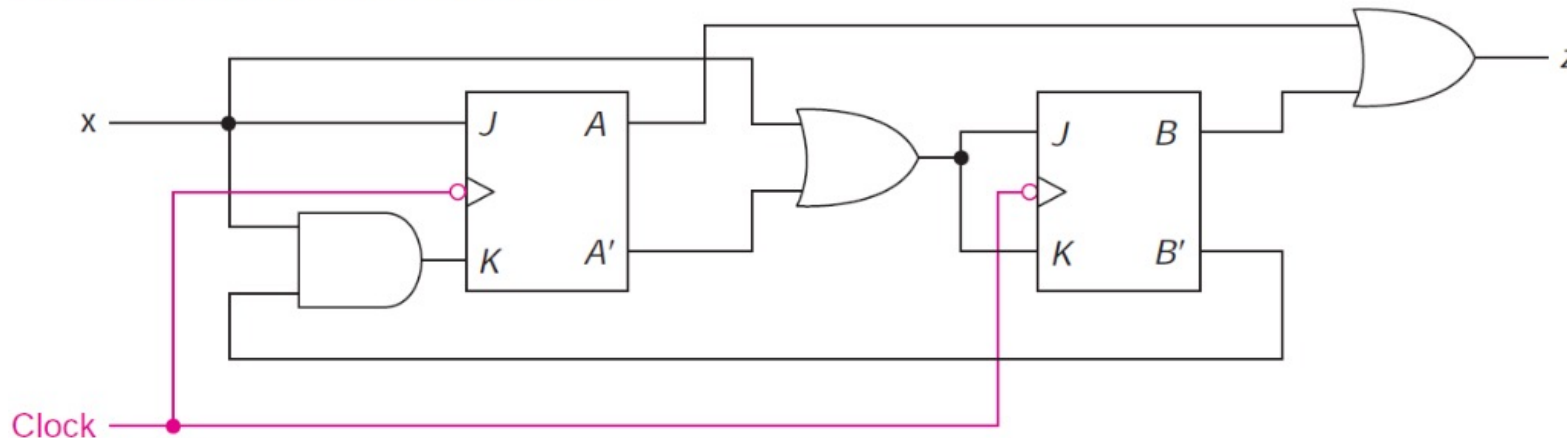


Example cont.

We have

- x is input to circuit
- JA and KA are inputs to the first FF and JB and KB are inputs to the second FF
- A and B are outputs of first and second FF respectively
- z is output of the entire circuit

Figure 6.24 A Moore model circuit.



Example cont.

- Equations for FF inputs and output:

$$J_A = x, \quad K_A = xB', \quad J_B = x + A', \quad K_B = x + A', \quad z = A + B$$

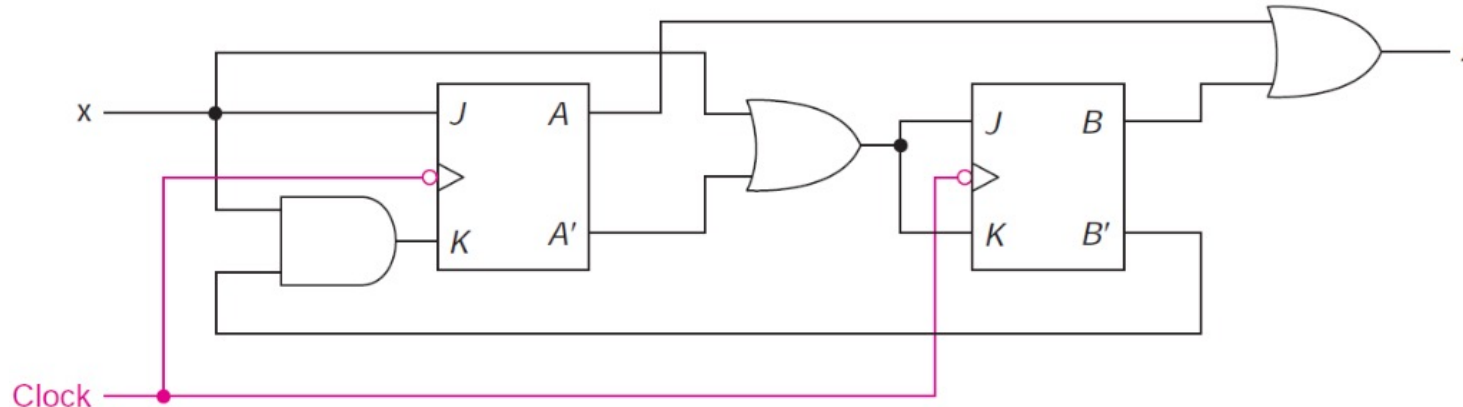
- For table, shall use behavioral equations of JK -FFs

$$q_1^\star = JQ' + K'Q$$

$$A^\star = J_A A' + K'_A A = xA' + (xB')'A = xA' + x'A + AB$$

$$B^\star = J_B B' + K'_B B = (x+A')B' + (x+A')'B = xB' + A'B' + x'AB$$

Figure 6.24 A Moore model circuit.



Example cont.

- For table, shall use behavioral equations of JK -FFs

$$- A^{\star} = xA' + x'A + AB$$

$$- B^{\star} = xB' + A'B' + x'AB$$

$$- x = 0$$

$$A^{\star} = A + AB = A$$

$$B^{\star} = A'B' + AB$$

$$- x = 1$$

$$A^{\star} = A' + AB = A' + B$$

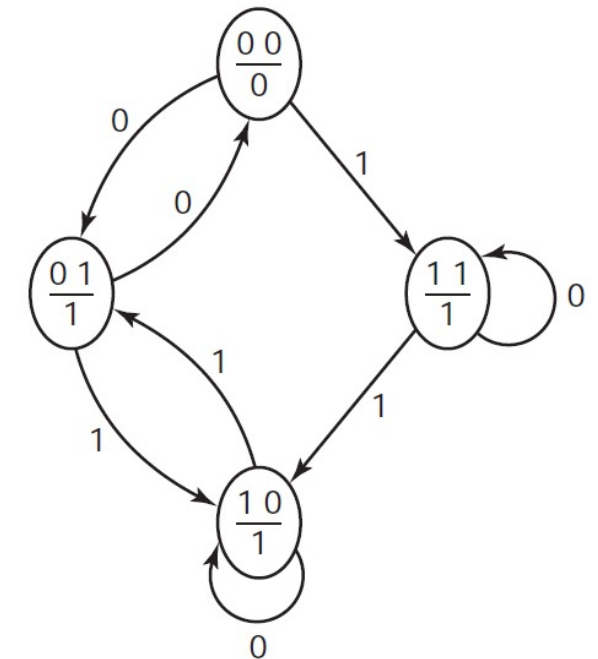
$$B^{\star} = B' + A'B' = B'$$

- Using the equations, we get the shown state table
- State diagram (**Moore model**) is also shown

Table 6.9c Completed state table.

$A \ B$	$A^{\star} \ B^{\star}$		z
	$x = 0$	$x = 1$	
0 0	0 1	1 1	0
0 1	0 0	1 0	1
1 0	1 0	0 1	1
1 1	1 1	1 0	1

Figure 6.26 State diagram for Table 6.9.



Example cont.

We will produce a timing trace and a timing diagram if we are given the input x and the initial state

- Suppose values of x are 0010110 and initial state is $AB = 00$; timing trace is

Trace 6.4 Trace for Table 6.9.

x	0	0	1	0	1	1	0	
A	0	0	0	1	1	1	0	0
B	0	1	0	1	1	0	1	0
								1
z	0	1	0	1	1	1	1	0
								1

Table 6.9c Completed state table.

$A \ B$	$A^* \ B^*$		z
	$x = 0$	$x = 1$	
0 0	0 1	1 1	0
0 1	0 0	1 0	1
1 0	1 0	0 1	1
1 1	1 1	1 0	1

Example cont.

- We can determine the output and value of B for one more clock time
- From state 00, next state is either 01 or 11, both of which have $B = 1$ and a 1 output
 - Cannot go any further

Trace 6.4 Trace for Table 6.9.

x	0	0	1	0	1	1	0	
A	0	0	0	1	1	1	0	0
B	0	1	0	1	1	0	1	0
z	0	1	0	1	1	1	1	0

Table 6.9c Completed state table.

$A \ B$	$A^* \ B^*$		z
	$x = 0$	$x = 1$	
0 0	0 1	1 1	0
0 1	0 0	1 0	1
1 0	1 0	0 1	1
1 1	1 1	1 0	1

Example cont.

Timing diagram for same system with same input sequence is shown

- Must look at values of A , B , and x (0010110) just before trailing edge
 - From that, we can determine what values for A and B must be during next clock period
- At any time we know A and B , we can determine z

Figure 6.25 Timing diagram for Table 6.9.

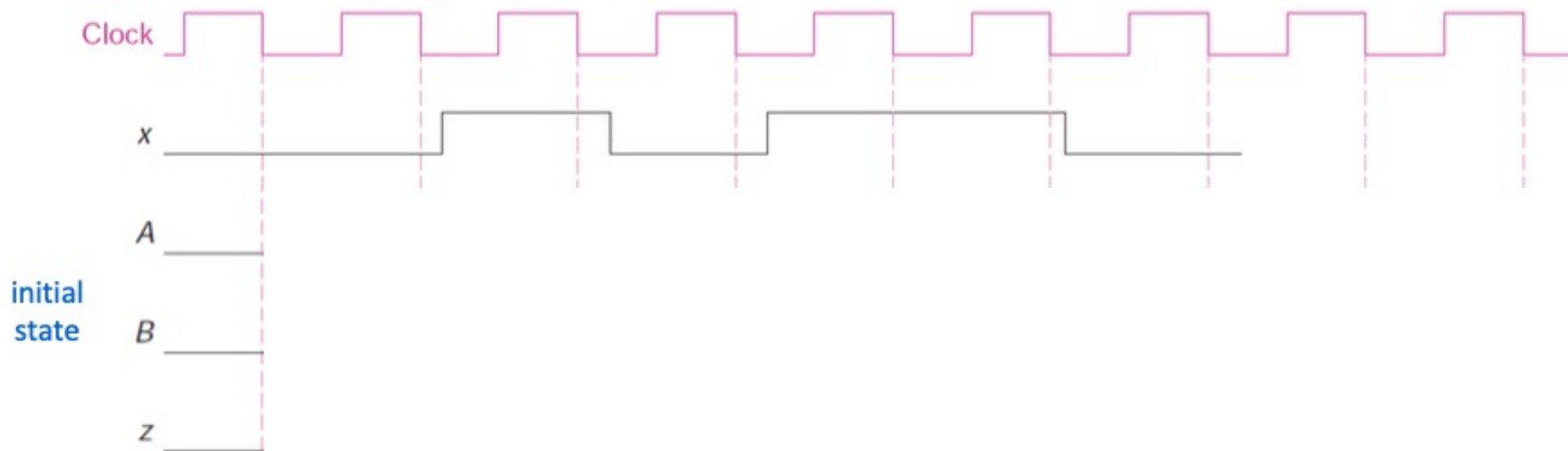


Table 6.9c Completed state table.

$A \ B$	$A^* \ B^*$		z
	$x = 0$	$x = 1$	
0 0	0 1	1 1	0
0 1	0 0	1 0	1
1 0	1 0	0 1	1
1 1	1 1	1 0	1

Example cont.

Timing diagram for same system with same input sequence is shown

- Must look at values of A , B , and x just before trailing edge;
 - From that, we can determine what values for A and B must be during next clock period
- At any time we know A and B , we can determine z

Figure 6.25 Timing diagram for Table 6.9.

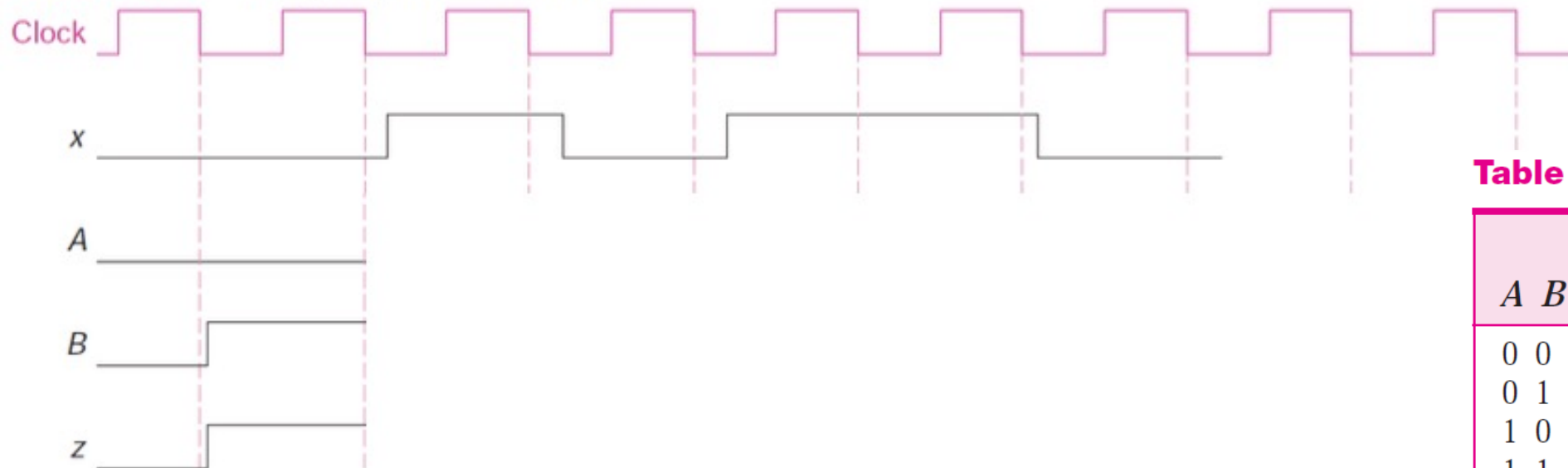


Table 6.9c Completed state table.

A B	$A^* B^*$		z
	$x = 0$	$x = 1$	
0 0	0 1	1 1	0
0 1	0 0	1 0	1
1 0	1 0	0 1	1
1 1	1 1	1 0	1

Example cont.

Timing diagram for same system with same input sequence is shown

- Must look at values of A , B , and x just before trailing edge;
 - From that, we can determine what values for A and B must be during next clock period
- At any time we know A and B , we can determine z

Figure 6.25 Timing diagram for Table 6.9.

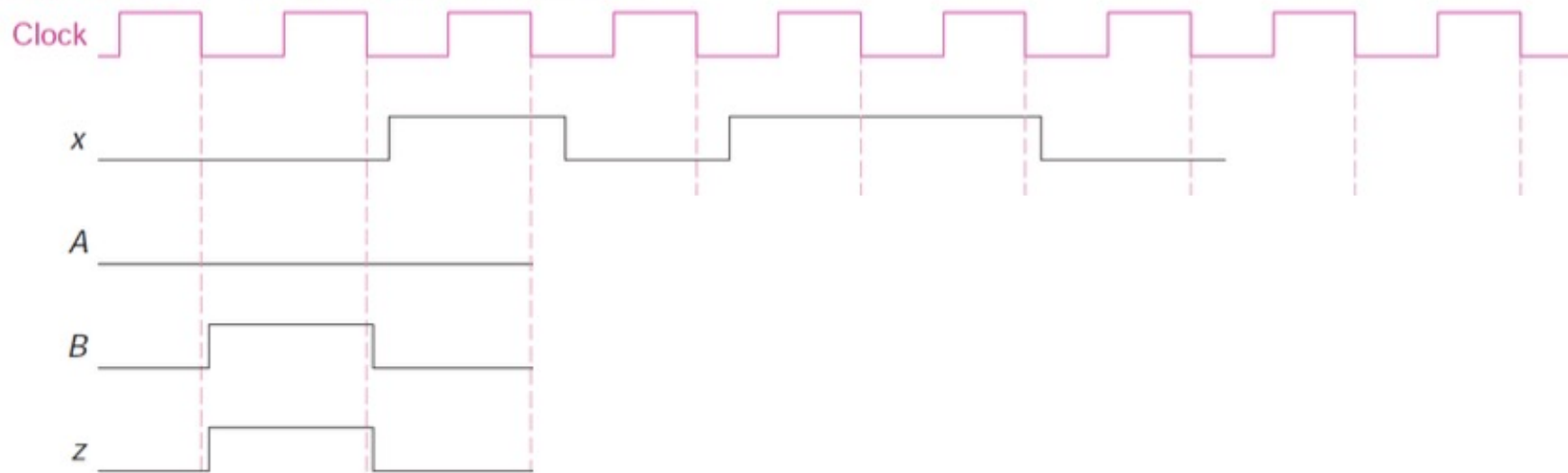


Table 6.9c Completed state table.

$A \ B$	$A^* \ B^*$		z
	$x = 0$	$x = 1$	
0 0	0 1	1 1	0
0 1	0 0	1 0	1
1 0	1 0	0 1	1
1 1	1 1	1 0	1

Example cont.

Timing diagram for same system with same input sequence is shown

- Must look at values of A , B , and x just before trailing edge;
 - From that, we can determine what values for A and B must be during next clock period
- At any time we know A and B , we can determine z

Figure 6.25 Timing diagram for Table 6.9.

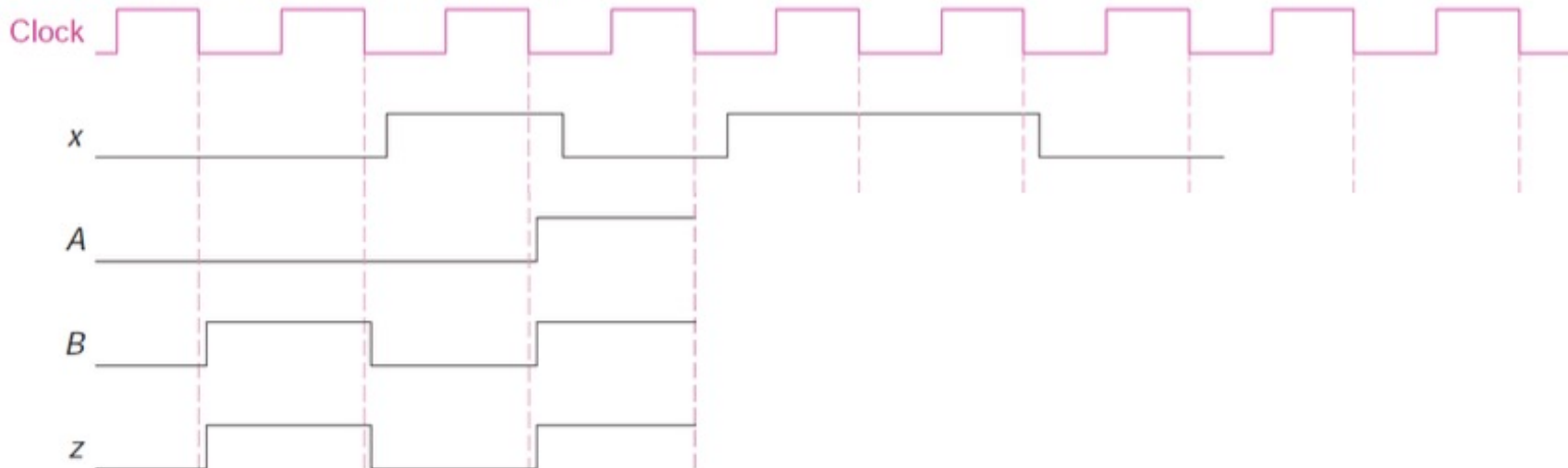


Table 6.9c Completed state table.

A B	$A^* B^*$		z
	$x = 0$	$x = 1$	
0 0	0 1	1 1	0
0 1	0 0	1 0	1
1 0	1 0	0 1	1
1 1	1 1	1 0	1

Example cont.

Timing diagram for same system with same input sequence is shown

- Must look at values of A , B , and x just before trailing edge;
 - From that, we can determine what values for A and B must be during next clock period
- At any time we know A and B , we can determine z

Figure 6.25 Timing diagram for Table 6.9.

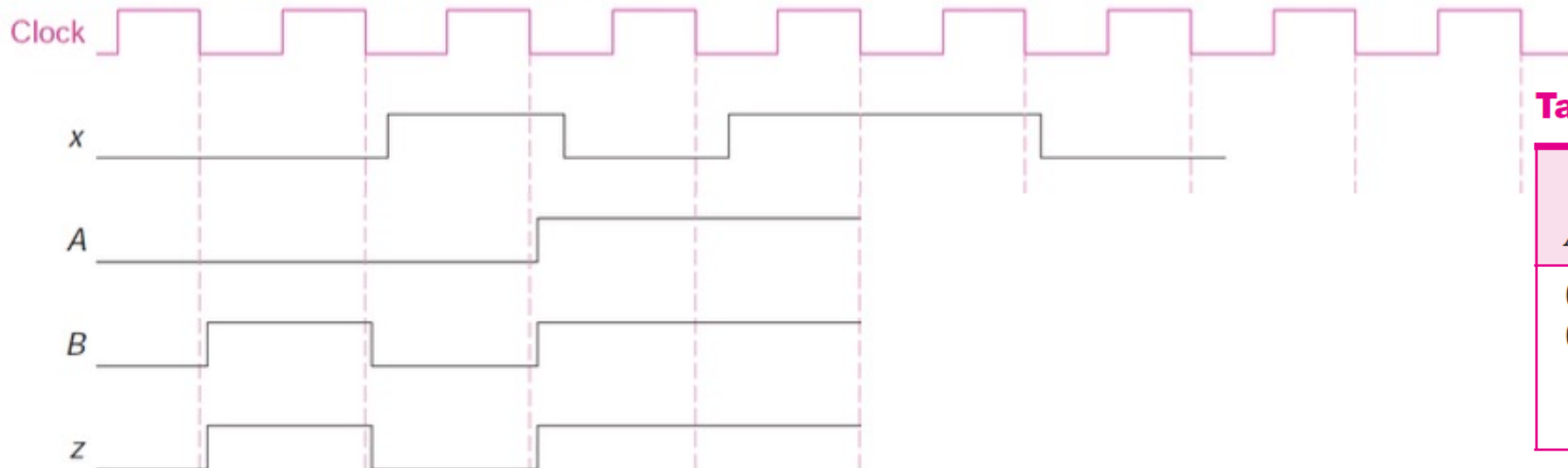


Table 6.9c Completed state table.

A B	$A^* B^*$		z
	$x = 0$	$x = 1$	
0 0	0 1	1 1	0
0 1	0 0	1 0	1
1 0	1 0	0 1	1
1 1	1 1	1 0	1

Example cont.

Timing diagram for same system with same input sequence is shown

- Must look at values of A , B , and x just before trailing edge;
 - From that, we can determine what values for A and B must be during next clock period
- At any time we know A and B , we can determine z

Figure 6.25 Timing diagram for Table 6.9.

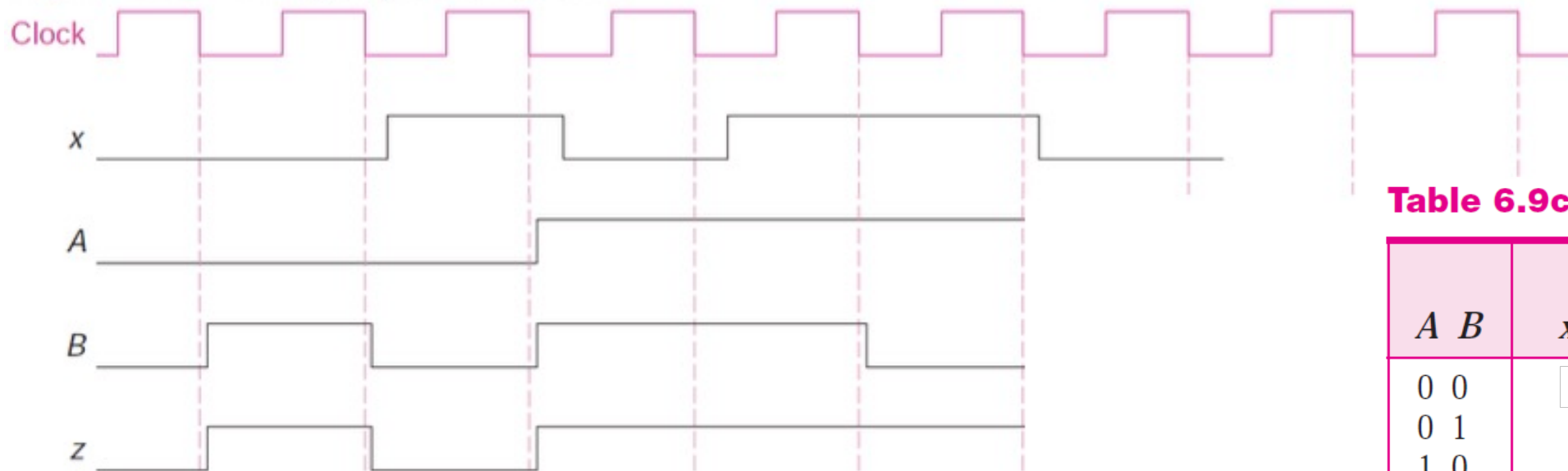


Table 6.9c Completed state table.

A B	$A^* B^*$		z
	$x = 0$	$x = 1$	
0 0	0 1	1 1	0
0 1	0 0	1 0	1
1 0	1 0	0 1	1
1 1	1 1	1 0	1

Example cont.

Timing diagram for same system with same input sequence is shown

- Must look at values of A , B , and x just before trailing edge;
 - From that, we can determine what values for A and B must be during next clock period
- At any time we know A and B , we can determine z

Figure 6.25 Timing diagram for Table 6.9.

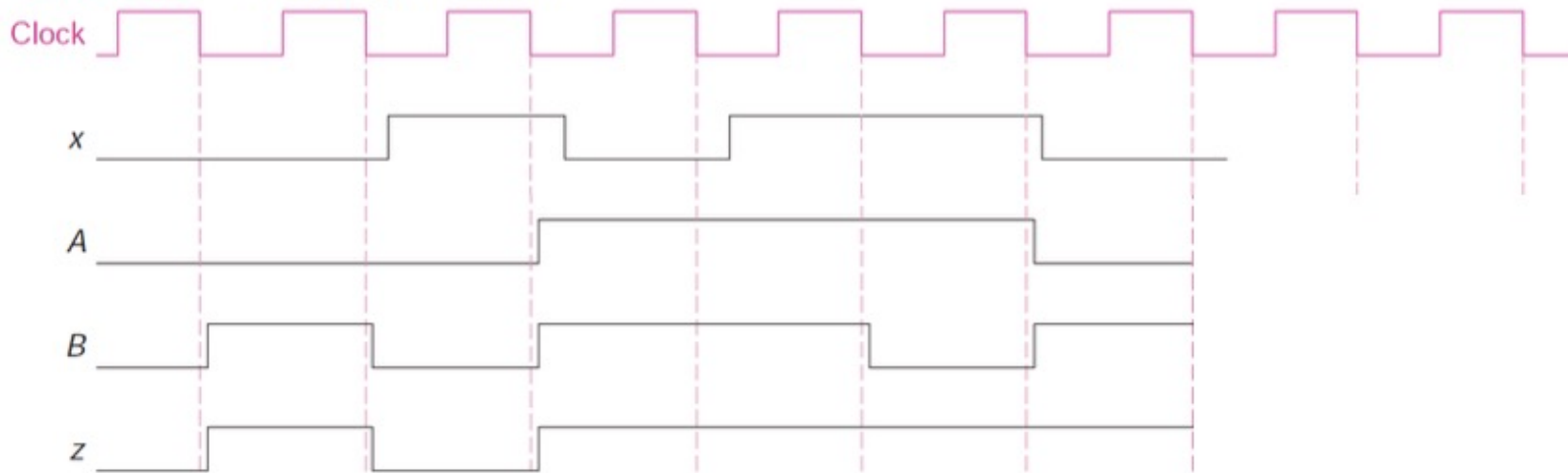


Table 6.9c Completed state table.

A B	$A^* B^*$		z
	$x = 0$	$x = 1$	
0 0	0 1	1 1	0
0 1	0 0	1 0	1
1 0	1 0	0 1	1
1 1	1 1	1 0	1

Example cont.

Timing diagram for same system with same input sequence is shown

- Must look at values of A , B , and x just before trailing edge;
 - From that, we can determine what values for A and B must be during next clock period
- At any time we know A and B , we can determine z

Figure 6.25 Timing diagram for Table 6.9.

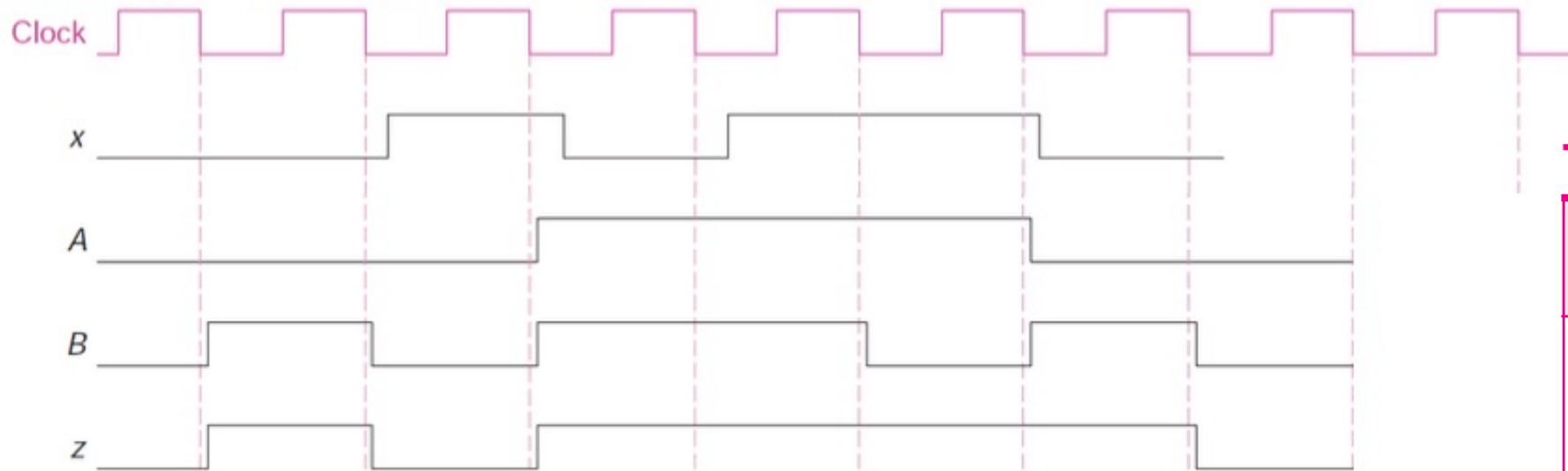


Table 6.9c Completed state table.

A B	$A^* B^*$		z
	$x = 0$	$x = 1$	
0 0	0 1	1 1	0
0 1	0 0	1 0	1
1 0	1 0	0 1	1
1 1	1 1	1 0	1

Example cont.

Timing diagram for same system with same input sequence is shown

- Must look at values of A , B , and x just before trailing edge;
 - From that, we can determine what values for A and B must be during next clock period
- At any time we know A and B , we can determine z

Figure 6.25 Timing diagram for Table 6.9.

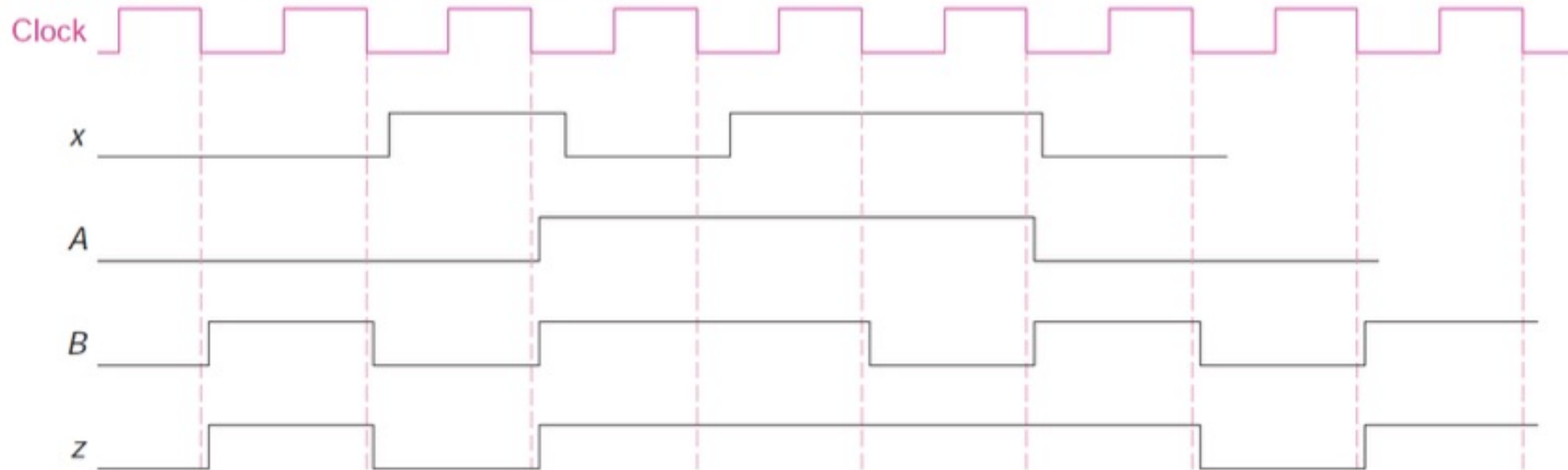


Table 6.9c Completed state table.

$A \ B$	$A^* \ B^*$		z
	$x = 0$	$x = 1$	
0 0	0 1	1 1	0
0 1	0 0	1 0	1
1 0	1 0	0 1	1
1 1	1 1	1 0	1

Analysis of Sequential Systems (Mealy Model)

- In some systems, **output depends on**
 - **present input**, as well as
 - **state**
- From a circuit point of view, that just means that z is a function of x , as well as the state variables
- This type of circuit is referred to as a **Mealy model**
 - Written: *state i/o*

Example: Mealy Model

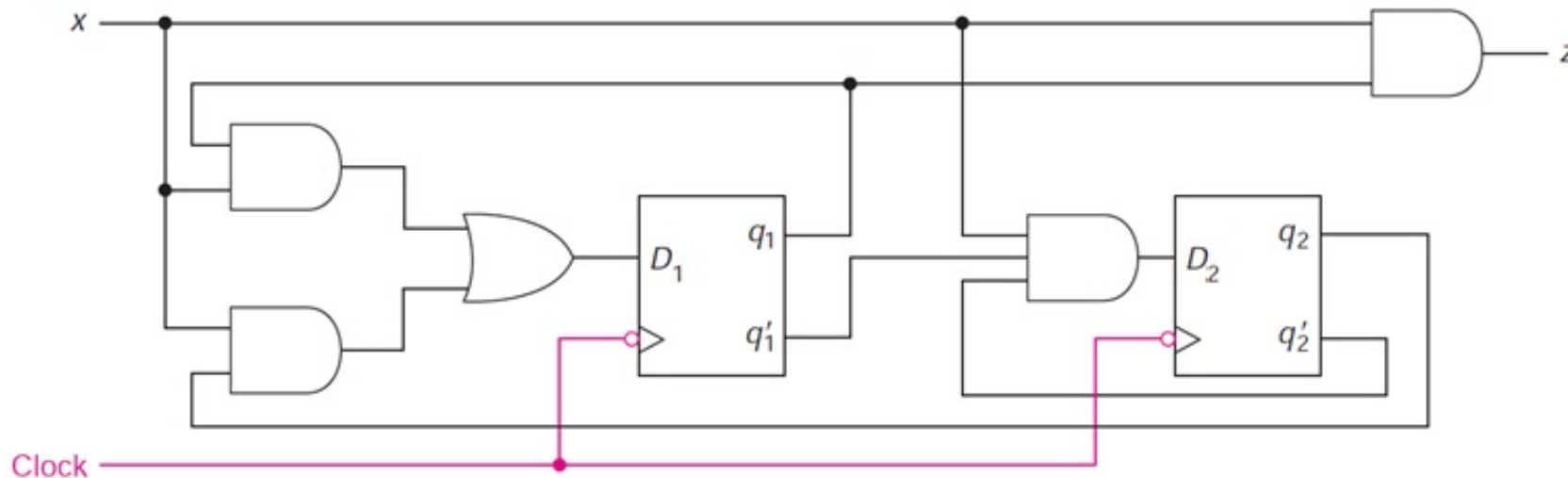
- Consider the following circuit with two D flip flops
- FF input and the output equations are:

$$D_1 = xq_1 + xq_2$$

$$D_2 = xq_1'q_2'$$

$$z = xq_1$$

Figure 6.27 A Mealy model.



Example cont.

With D flip flops, $q^{\star}=D$

$$q_1^{\star}=xq_1+xq_2 \qquad q_2^{\star}=xq_1'q_2' \qquad z=xq_1$$

- From the equations, obtain the state table
 - Notice that we need two output columns, one for $x=0$ and one for $x=1$

Table 6.10 State table for the Mealy system.

q	$q^{\star}$		z	
	$x=0$	$x=1$	$x=0$	$x=1$
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1
1 1	0 0	1 0	0	1

Example cont.

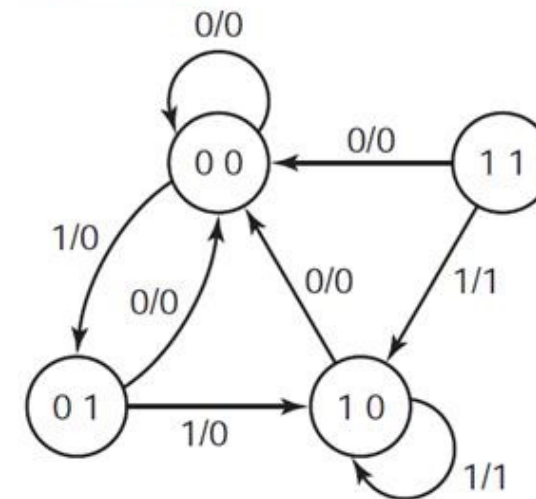
State diagram

- Note that state 11 is never reached; this problem really has only three states
- When system is first turned on, it could start in state 11
 - But after first clock, it will leave that state and never return

Table 6.10 State table for the Mealy system.

q	q^*		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1
1 1	0 0	1 0	0	1

Figure 6.28 State diagram for a Mealy model.



Example cont.

Timing trace for the given system is shown; notices:

- Even initial state is unknown (?), 0 input forces system to state 00 at next clock time
- Output is known for two clock periods after input is not
 - Since system cannot reach state 10 (only state for which there is a 1 output) any sooner than that
- Present state and present input determine *present* output and *next* state

Table 6.10 State table for the Mealy system.

q	q^*		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1
1 1	0 0	1 0	0	1

Trace 6.5 Mealy model timing.

x	0	1	1	0	1	1	1	1	0	
q_1	?	0	0	1	0	0	1	1	1	0
q_2	?	0	1	0	0	1	0	0	0	0
z	0	0	0	0	0	0	1	1	0	0

Example cont.

Timing diagram is shown; it illustrates a peculiarity of **Mealy** systems

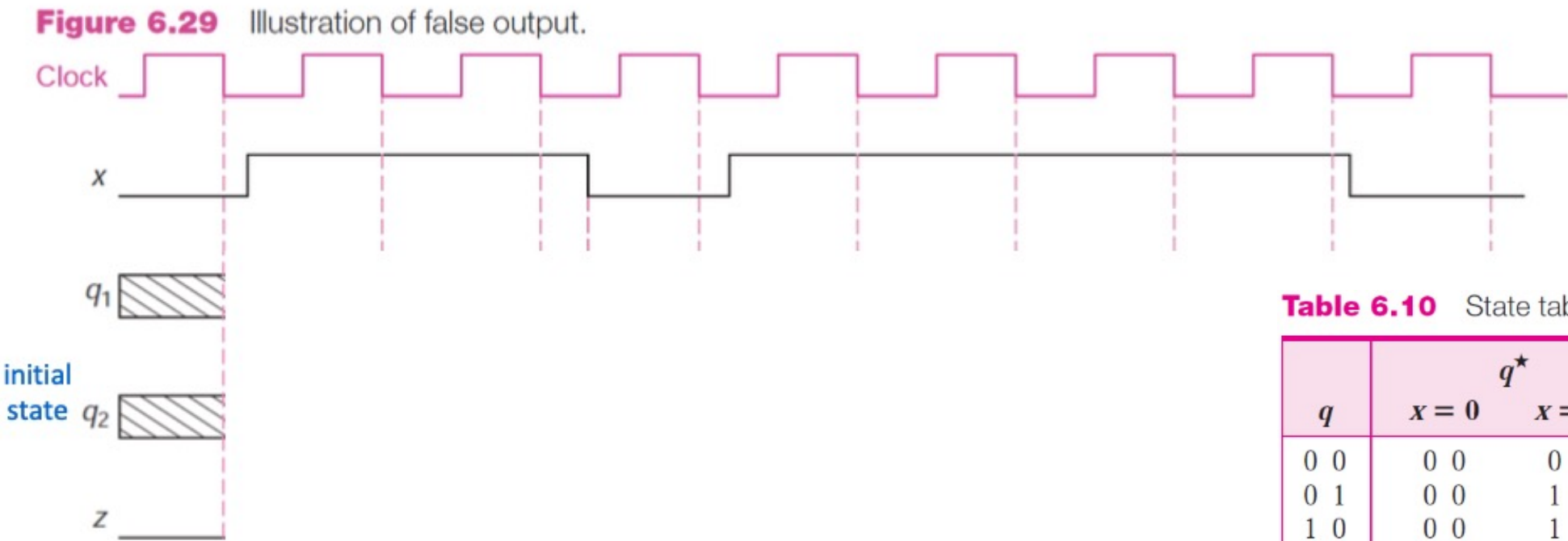


Table 6.10 State table for the Mealy system.

<i>q</i>	<i>q</i> [★]		<i>z</i>	
	<i>x</i> = 0	<i>x</i> = 1	<i>x</i> = 0	<i>x</i> = 1
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1
1 1	0 0	1 0	0	1

Example cont.

Timing diagram is shown; it illustrates a peculiarity of Mealy systems

Figure 6.29 Illustration of false output.

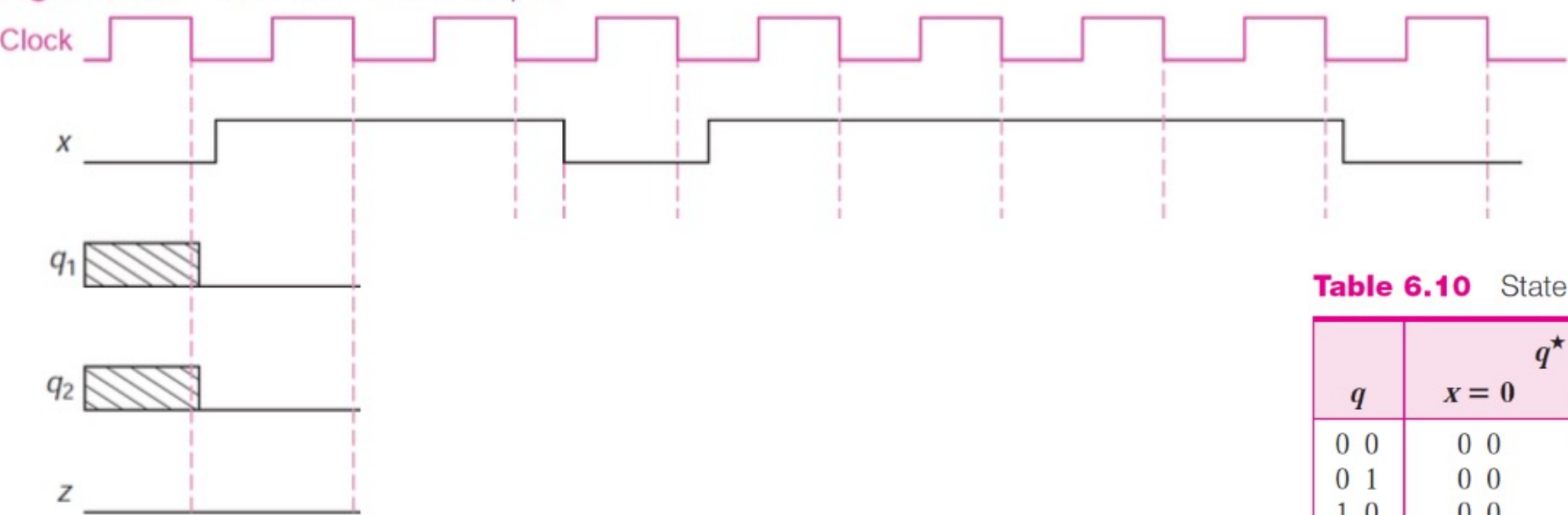


Table 6.10 State table for the Mealy system.

<i>q</i>	<i>q</i> [★]		<i>z</i>	
	<i>x</i> = 0	<i>x</i> = 1	<i>x</i> = 0	<i>x</i> = 1
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1
1 1	0 0	1 0	0	1

Example cont.

Timing diagram is shown; it illustrates a peculiarity of **Mealy** systems

Figure 6.29 Illustration of false output.

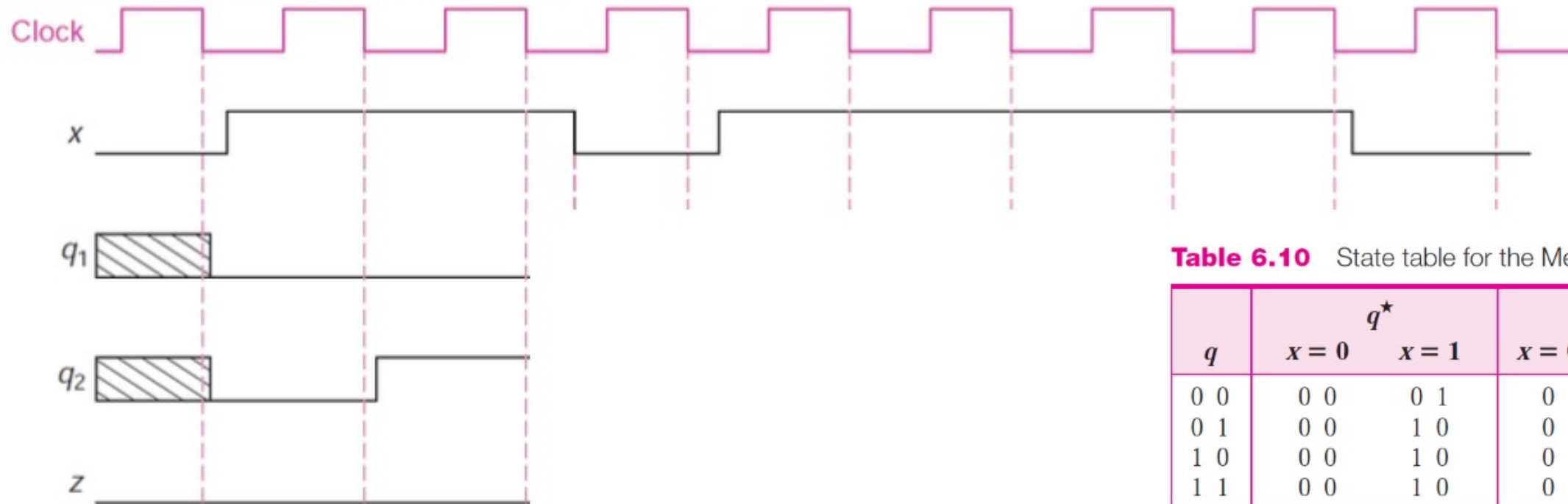


Table 6.10 State table for the Mealy system.

q	q^*		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1
1 1	0 0	1 0	0	1

Example cont.

Timing diagram is shown; it illustrates a peculiarity of **Mealy** systems

Figure 6.29 Illustration of false output.

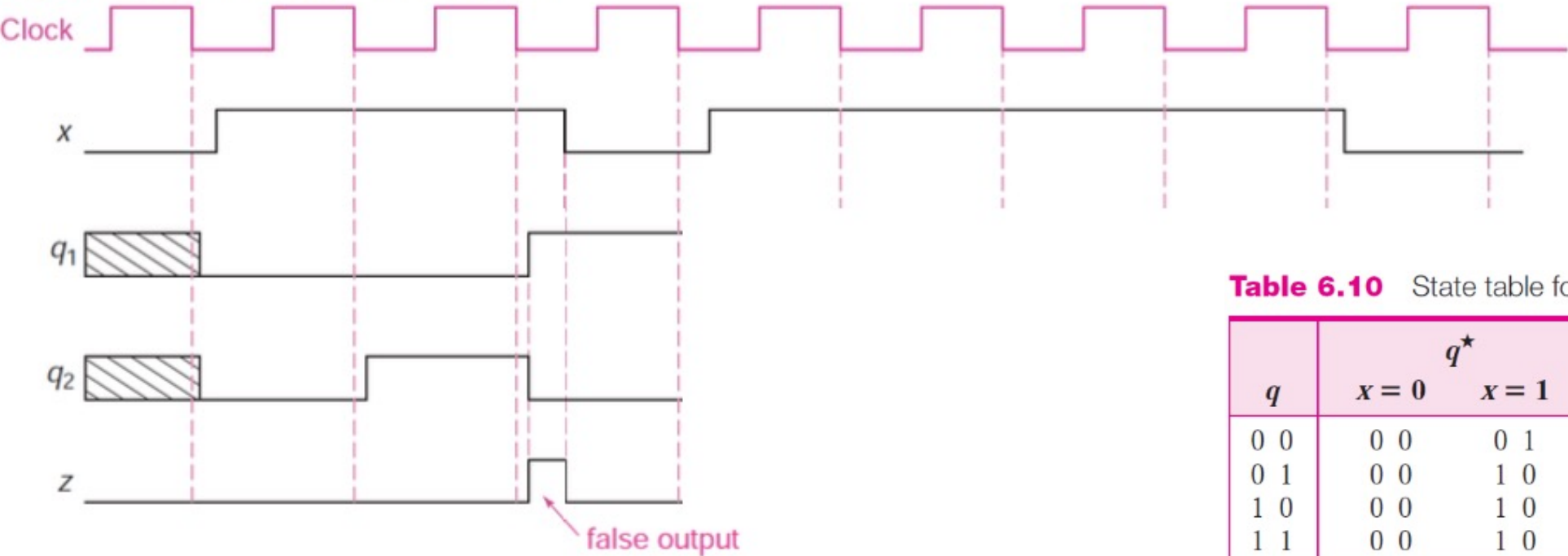


Table 6.10 State table for the Mealy system.

<i>q</i>	<i>q</i> [★]		<i>z</i>	
	<i>x</i> = 0	<i>x</i> = 1	<i>x</i> = 0	<i>x</i> = 1
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1
1 1	0 0	1 0	0	1

Example cont.

Timing diagram is shown; it illustrates a peculiarity of **Mealy** systems

Figure 6.29 Illustration of false output.

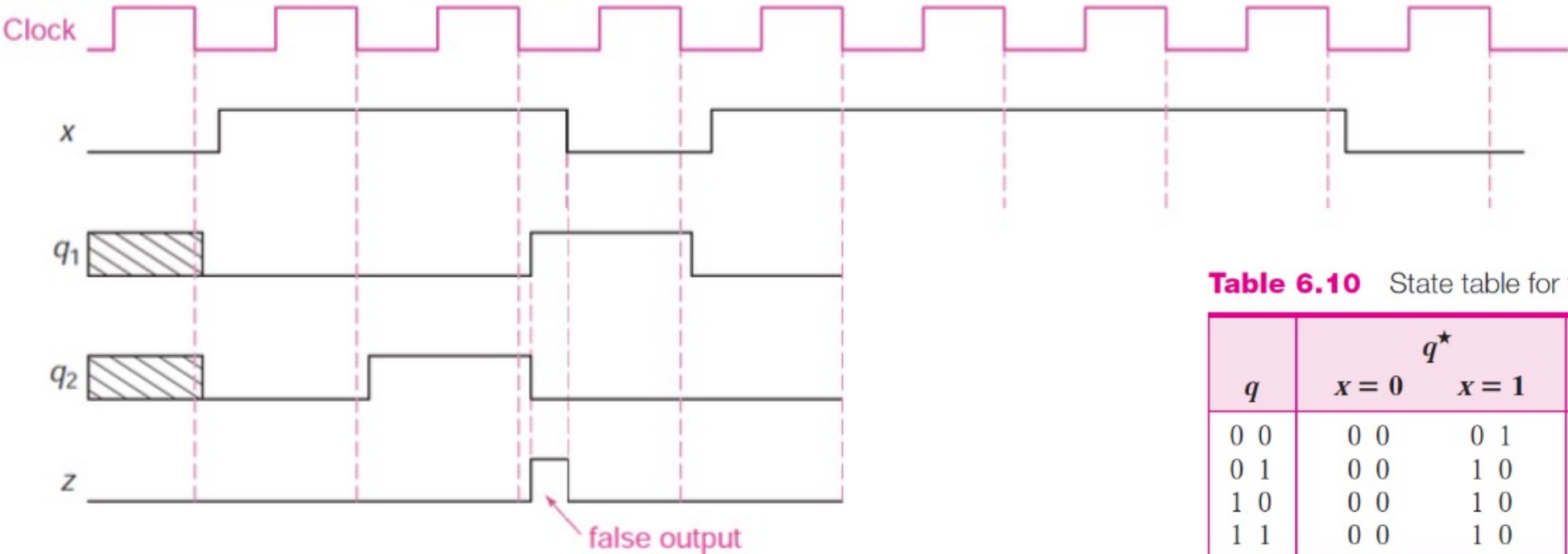


Table 6.10 State table for the Mealy system.

<i>q</i>	<i>q</i> [★]		<i>z</i>	
	<i>x</i> = 0	<i>x</i> = 1	<i>x</i> = 0	<i>x</i> = 1
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1
1 1	0 0	1 0	0	1

Example cont.

Timing diagram is shown; it illustrates a peculiarity of **Mealy** systems

Figure 6.29 Illustration of false output.

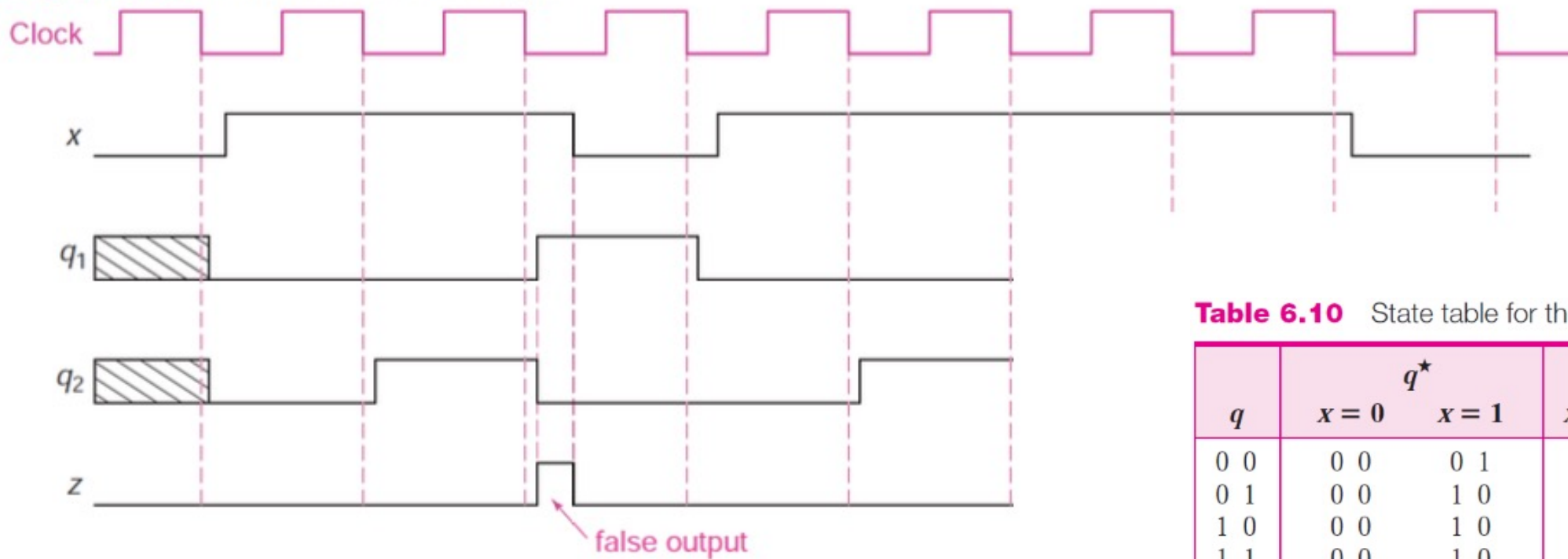


Table 6.10 State table for the Mealy system.

q	q^*		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1
1 1	0 0	1 0	0	1

Example cont.

Timing diagram is shown; it illustrates a peculiarity of **Mealy** systems

Figure 6.29 Illustration of false output.

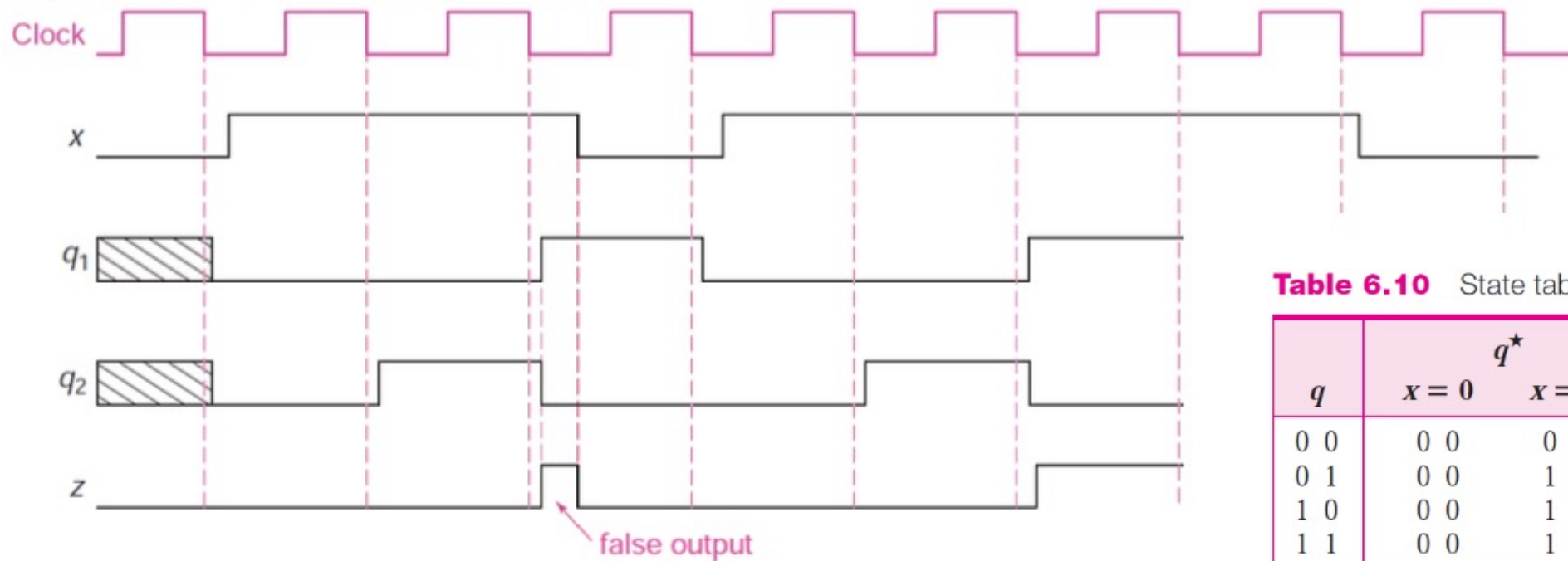


Table 6.10 State table for the Mealy system.

q	q^*		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1
1 1	0 0	1 0	0	1

Example cont.

Timing diagram is shown; it illustrates a peculiarity of Mealy systems

Figure 6.29 Illustration of false output.

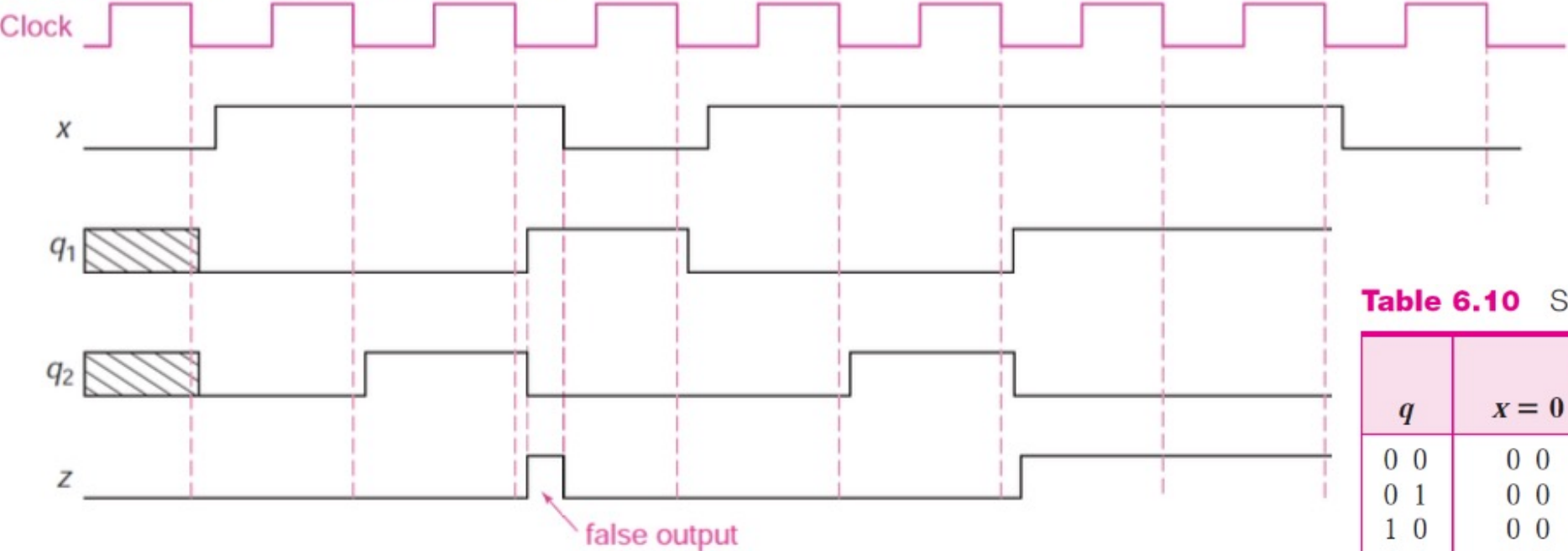


Table 6.10 State table for the Mealy system.

q	q^*		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1
1 1	0 0	1 0	0	1

Example cont.

Timing diagram is shown; it illustrates a peculiarity of Mealy systems

Figure 6.29 Illustration of false output.

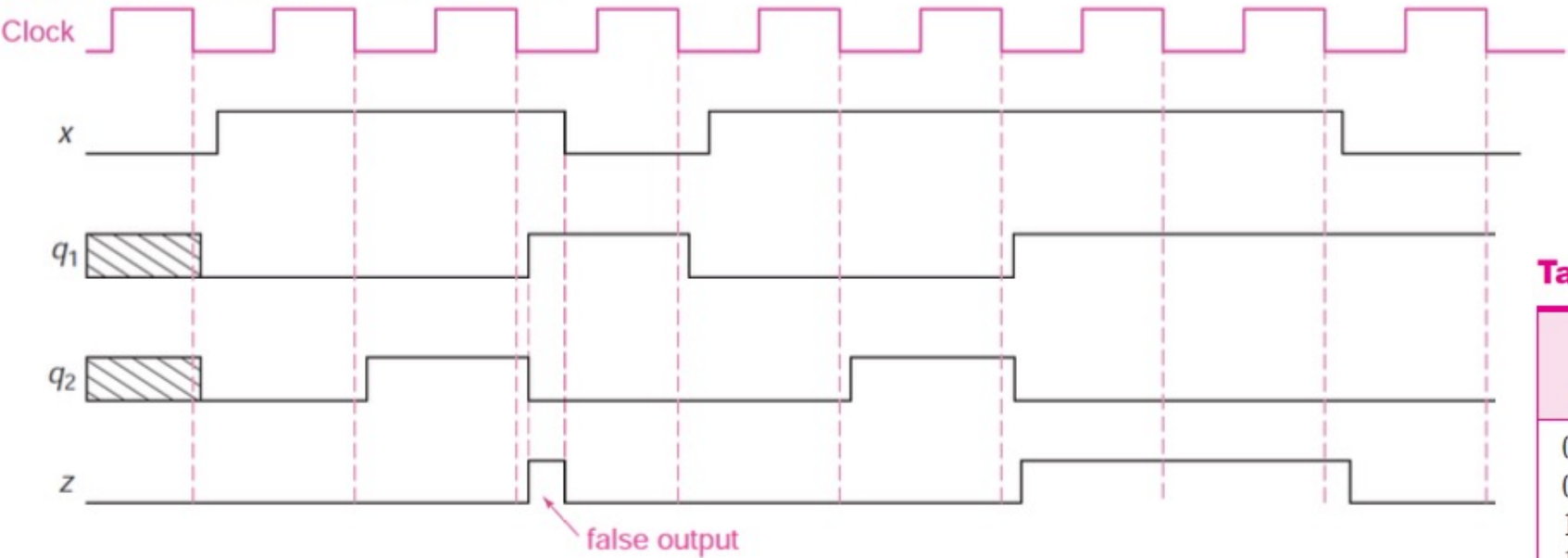


Table 6.10 State table for the Mealy system.

q	q^*		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1
1 1	0 0	1 0	0	1

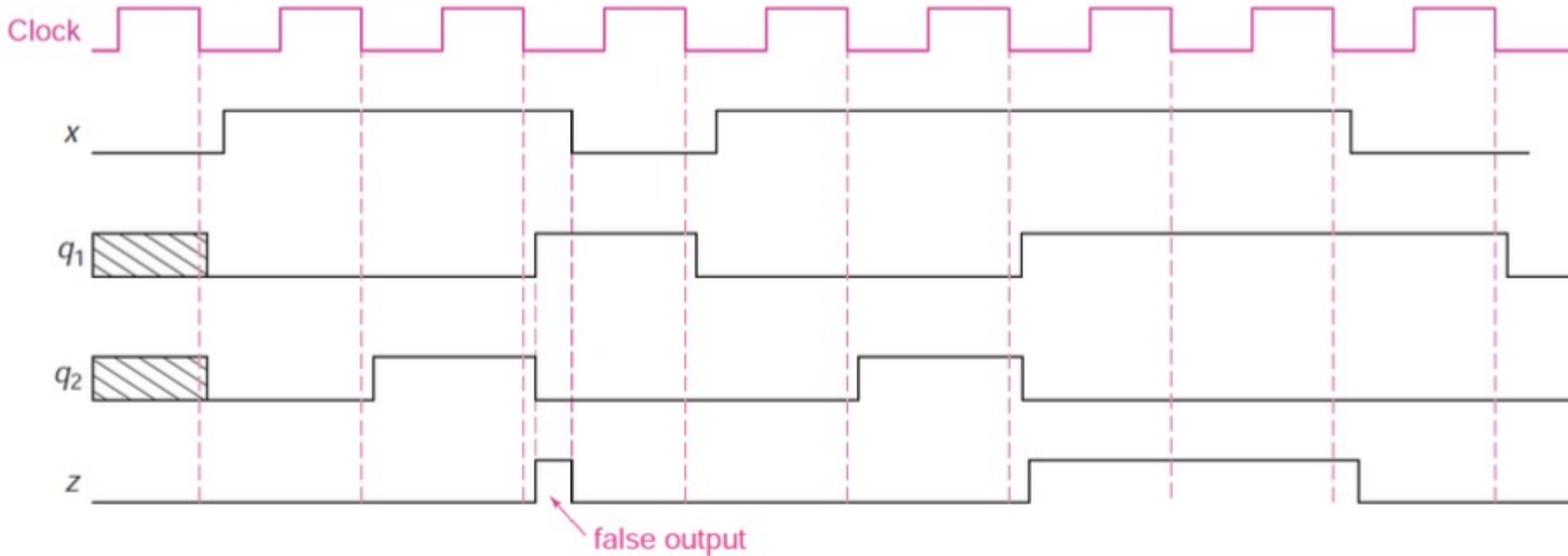
Example cont.

Timing diagram is shown; it illustrates a peculiarity of **Mealy** systems

Table 6.10 State table for the Mealy system.

q	q^*		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1
1 1	0 0	1 0	0	1

Figure 6.29 Illustration of false output.



Example cont.

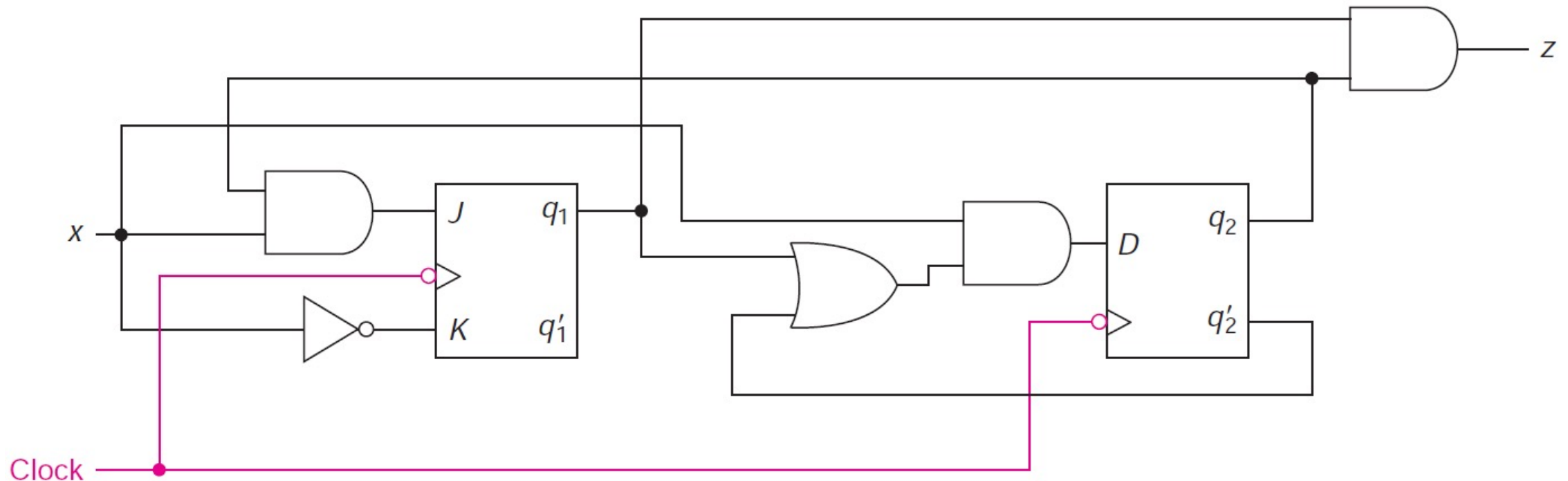
- Note that there is a *false output* (sometimes referred to as a *glitch*)
 - Output goes to 1 for a short period even though that is not indicated in the timing trace nor in the state table
- Output comes from combinational logic; it is just xq_1
- If input x does not change simultaneously with the trailing edge of clock (as here), and it remains 1 after q_1 goes to 1, output will go to 1
- But output indicated by state table or timing trace is based on value of q_1 at the time of the next clock

Example cont.

- False output is not usually important, since output of Mealy system is mainly of interest at clock times
 - Just before edge on which flip flops might change state
- It is often the case that system inputs change simultaneously with the trailing edge of the clock
- *Glitch* occurred only because x changed well after the flip flop change
- As change in x gets closer (in time) to changes in q (clock edge), glitch gets narrower and if x goes to 0 at the same time that q_1 goes to 1, false output disappears

Another Example

Consider the following circuit with one *JK* flip flop and one *D* flip flop



Example cont.

- Output, $z = q_1q_2$, is a function of the state (contents of FFs) and does not depend on the input x
 - Moore model
 - This system also produces an output of 1 if the input is 1 for three or more consecutive clock periods

- Input equations for the system are

$$J_1 = xq_2$$

$$K_1 = x'$$

$$D_2 = x(q_1 + q_2')$$

Example cont.

For this table, we shall use the behavioral equations of *JK*-flip flop and *D*-flip flop as:

$$q^{\star} = Jq' + K'q \qquad q^{\star} = D$$

$$\begin{aligned} q_1^{\star} &= J_1 q_1' + K_1' q_1 \\ &= x q_2 q_1' + (x')' q_1 = x q_2 q_1' + x q_1 \\ &= x(q_2 q_1' + q_1) = x(q_2 + q_1)(q_1' + q_1) \\ &= x(q_2 + q_1) \end{aligned}$$

$$q_2^{\star} = D_2 = x(q_1 + q_2')$$

Example cont.

From either approach, we get the following state table:

$$q_1^* = x(q_2 + q_1)$$

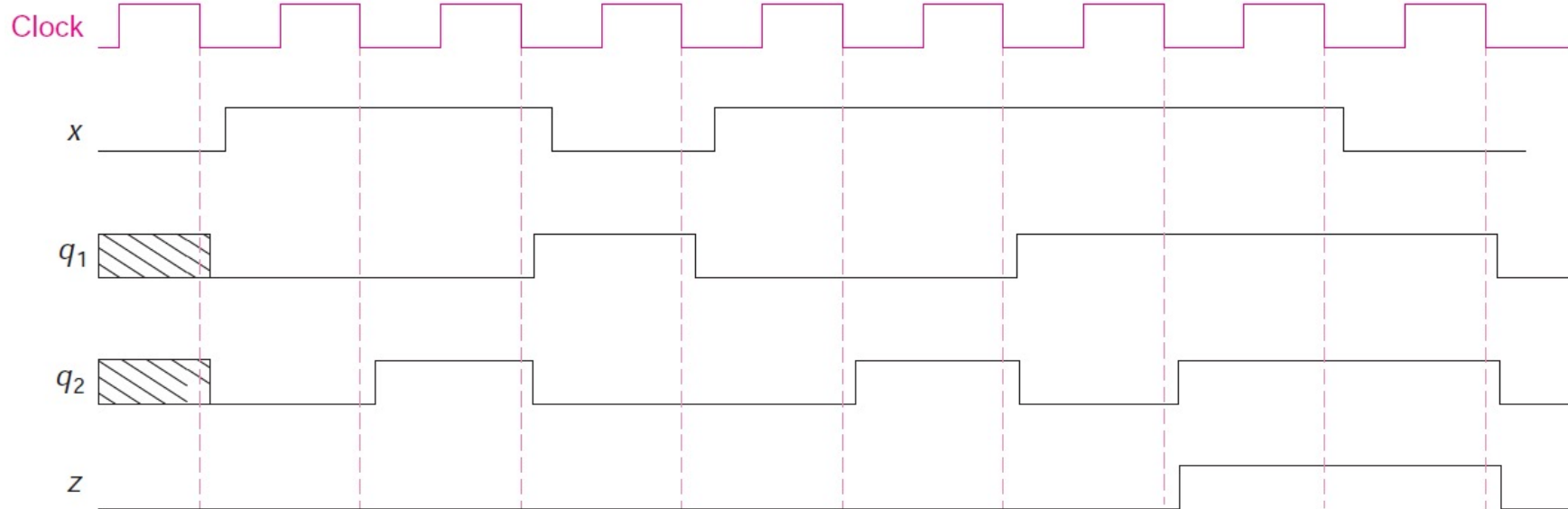
$$q_2^* = D_2 = x(q_1 + q_2')$$

$$z = q_1 q_2$$

$q_1 q_2$	$q_1^* q_2^*$		z
	$x = 0$	$x = 1$	
0 0	0 0	0 1	0
0 1	0 0	1 0	0
1 0	0 0	1 1	0
1 1	0 0	1 1	1

Example cont.

- State diagram (i.e. Moore model) circuit is left as an exercise
- Timing diagram for the same system is shown below



$q_1 q_2$	$q_1^* q_2^*$		z
	$x = 0$	$x = 1$	
0 0	0 0	0 1	0
0 1	0 0	1 0	0
1 0	0 0	1 1	0
1 1	0 0	1 1	1

Example cont.

- There is never a false output in a Moore model
 - Since output depends only on the state of the flip flops, and they all change simultaneously, on the trailing edge of the clock
- Output is valid for a whole clock period, from just after one negative- going transition to just after the next

Example cont.

- If the AND gate of previous example had a third input, x' , then $z = x'q_1q_2$ and this would be a Mealy model
- Next state of state table and of timing diagram would be unchanged
- There would be two columns in the state table as shown below

$q_1 q_2$	$q_1^* q_2^*$		z
	$x = 0$	$x = 1$	
0 0	0 0	0 1	0
0 1	0 0	1 0	0
1 0	0 0	1 1	0
1 1	0 0	1 1	1

$q_1 q_2$	$q_1^* q_2^*$		z	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 1	0	0
1 1	0 0	1 1	1	0

The End